Patrones multivariados del tráfico de red y detección de ataques en UNSW-NB15

Proyecto integrador

[Integrante 1]
[Integrante 2]
[Integrante 3]
[Integrante 4]

Universidad Católica de Córdoba
Facultad de [completar]
Carrera de [completar]
Sede Río Cuarto

Materia: Estadística Multivariada
Docente a cargo: Prof. Silvana Malpassi

Fecha de entrega: [día] de [mes] de 2026

Índice

1. Introducción **3**

2. Datos **3**

3. Análisis de componentes principales **5**

4. Análisis factorial **6**

5. Análisis de conglomerados **8**

6. Análisis de correspondencias simple y múltiples **10**
6.1. Construcción de X_cat 10
6.2. Análisis de correspondencias simple: `tipo_trafico` × `state_g` 10
6.3. Análisis de correspondencias múltiples 11

7. Regresión logística **12**

8. Integración y conclusiones **13**

Referencias bibliográficas **15**

A. Anexo: código fuente **16**

Índice de figuras

1. Correlaciones, diagramas de caja y MSA de X_num 4
2. Histogramas de variables de X_num en escala logarítmica 5
3. Codo y biplot del ACP 6
4. Análisis paralelo del AF 7
5. Codo y silhouette para elegir k 9
6. Perfiles por cluster y dendrograma de variables 9
7. Mapa de categorías del ACM 11
8. Clusters suplementarios y eta^2 del ACM 12
9. Curva ROC y matriz de confusión de M2 13

Índice de tablas

1. Variables de X_num: significado, unidad y resumen numérico 4
2. Cargas y calidad de representación del ACP 6
3. Matriz de patrón, comunalidades y unicidades del AF 7
4. Comparación entre ACP y AF 8
5. Perfil de los seis clusters 10
6. Variables y categorías finales de X_cat 10
7. Contribuciones y cos2 de las categorías del ACM 11
8. Integración de las técnicas 14

1. Introducción

La detección de intrusiones busca identificar, a partir del tráfico de una red, las conexiones que forman parte de un ataque. Una de las bases etiquetadas más difundidas es UNSW-NB15, creada por Moustafa and Slay (2015) en UNSW Canberra, que combina tráfico normal con nueve tipos de ataque generados de forma sintética (Moustafa and Slay, 2016). Cada fila es un flujo (una conversación entre un origen y un destino) descrito por variables de volumen, tiempo, conexión y contexto, con muchas numéricas correlacionadas, varias categóricas y una etiqueta externa (normal o ataque) con la cual contrastar los patrones; el ACP, el AF y los conglomerados no la usan.

Nuestra pregunta guía es: *¿qué patrones de volumen, temporización y repetición de conexiones caracterizan a los flujos de UNSW-NB15, y en qué medida esos patrones, junto con el protocolo, el servicio y el estado de la conexión, distinguen el tráfico normal del de ataque al punto de permitir estimar la probabilidad de que un flujo sea un ataque?* Las hipótesis se apoyan en el diseño de la base y en el dominio; como ya habíamos explorado los datos al fijarlas, las contrastamos al final, incluidas las que no se cumplan.

- **H1.** Las 12 variables numéricas forman tres bloques correlacionados entre sí: volumen, temporización y repetición de conexiones; el ACP, el AF y el cluster de variables deberían recuperarlos.
- **H2.** Existen grupos de flujos definidos sobre todo por la forma de la conexión; separan parcialmente lo normal de lo atacante, pero no los tipos de ataque entre sí.
- **H3.** El ataque se asocia con conexiones incompletas (estado INT) y sin respuesta del destino; lo normal, con conexiones cerradas (FIN, CON) y servicios identificados.
- **H4.** Un modelo logístico con los ejes del ACP, el protocolo, el servicio y el estado alcanza un AUC mayor a 0,80 en datos de validación.

2. Datos

Origen y muestra. Trabajamos con UNSW-NB15 (Moustafa and Slay, 2015, 2016), de uso académico citando a sus autores, con R (R Core Team, 2024) y los paquetes `psych` y `ca` (Nenadić and Greenacre, 2007). El tráfico normal es realista, pero los ataques son sintéticos (en las variables, `s` indica el origen y `d` el destino). Del archivo de entrenamiento (175.341 flujos) tomamos 5.000 únicos, con muestreo estratificado proporcional por `attack_cat`: 48,2 % normales y 51,8 % ataques (Exploits 18,4 %, Fuzzers 15,0 %, Reconnaissance 7,0 %, otros 11,5 %). Sin duplicados (eran el 38,6 % del original), las proporciones no son prevalencias reales.

Diccionario y variables de análisis. La Tabla 1 describe las 12 variables de X_num, agrupadas en los tres bloques esperados, con su unidad y su resumen numérico; el diccionario de las 68 columnas está en la hoja `diccionario` del Excel de la muestra. X_cat (Tabla 6, sección 6) tiene siete categóricas: cuatro existentes y tres numéricas discretizadas. `tipo_trafico`, `attack_cat` y `label` no construyen componentes, factores ni clusters: solo colorean o se cruzan después. Como las variables son positivas, muy asimétricas y de rangos amplios, analizamos $\log(x+1)$, que conserva los ceros y baja la asimetría de `sbytes` de 33,3 a 1,0 (Figura 2).

Tabla 1. Variables numéricas de X_num por bloque esperado, con unidad y resumen numérico en la unidad original (n = 5.000; el análisis usa $\log(x+1)$). Q1 y Q3 están en el repositorio. Fuente: elaboración propia con UNSW-NB15.

Variable	Significado	Unidad	Media	Desvío	Mín.	Mediana	Máx.
Volumen							
`spkts`	Paquetes enviados por el origen	paquetes	32,4	227	1	10	9.616
`sbytes`	Bytes enviados por el origen	bytes	16.918	296.287	46	918	12.965.233
`dbytes`	Bytes enviados por el destino	bytes	26.437	234.484	0	354	12.422.665
Temporización							
`dur`	Duración total del flujo	s	1,34	5,52	0	0,297	60
`sinpkt`	Tiempo entre paquetes del origen	ms	484	4.778	0	19,9	60.001
`sjit`	Jitter del origen	ms	5.636	39.960	0	1.186	934.346
`synack`	Tiempo SYN a SYN-ACK (0 si no es TCP)	s	0,032	0,046	0	0,0006	0,731
Repetición (conexiones entre las últimas 100 con...)							
`ct_srv_src`	igual servicio y origen	conexiones	5,09	6,33	1	3	63
`ct_srv_dst`	igual servicio y destino	conexiones	4,76	6,25	1	3	62
`ct_src_ltm`	igual origen	conexiones	3,81	4,81	1	2	50
`ct_src_dport_ltm`	igual origen y puerto destino	conexiones	1,95	3,34	1	1	50
`ct_dst_sport_ltm`	igual destino y puerto origen	conexiones	1,42	2,04	1	1	28

Calidad. Aplicamos 16 controles (faltantes, vacíos, duplicados por fila, identificador y perfil, valores negativos, no finitos o no enteros, TTL y ventana fuera de rango, `dur` = 0 con `spkts` >1, `sbytes` <`spkts`, indicadores no binarios y etiqueta inconsistente). La única incidencia es un flujo con `is_ftp_login` = 4, que conservamos porque esa variable no se usa. No imputamos los ceros estructurales: `synack` 29,6 %, `sjit` 25,9 % y `dbytes` 18,9 % (sin respuesta). Criterio de atípicos (Tukey, sobre la escala logarítmica): es atípico el valor fuera de $[Q_1 - 1{,}5\,\mathrm{RIC};\ Q_3 + 1{,}5\,\mathrm{RIC}]$, con RIC el rango intercuartílico. Según los diagramas de caja de la Figura 1, 1.715 flujos (34,3 %) tienen al menos una variable fuera de los límites, sobre todo `dbytes` (21,1 %; 945 son los ceros sin respuesta), `ct_src_dport_ltm` (11,6 %) y `ct_dst_sport_ltm` (9,3 %, con RIC nulo). Los conservamos: en ciberseguridad lo extremo puede ser el fenómeno de interés.

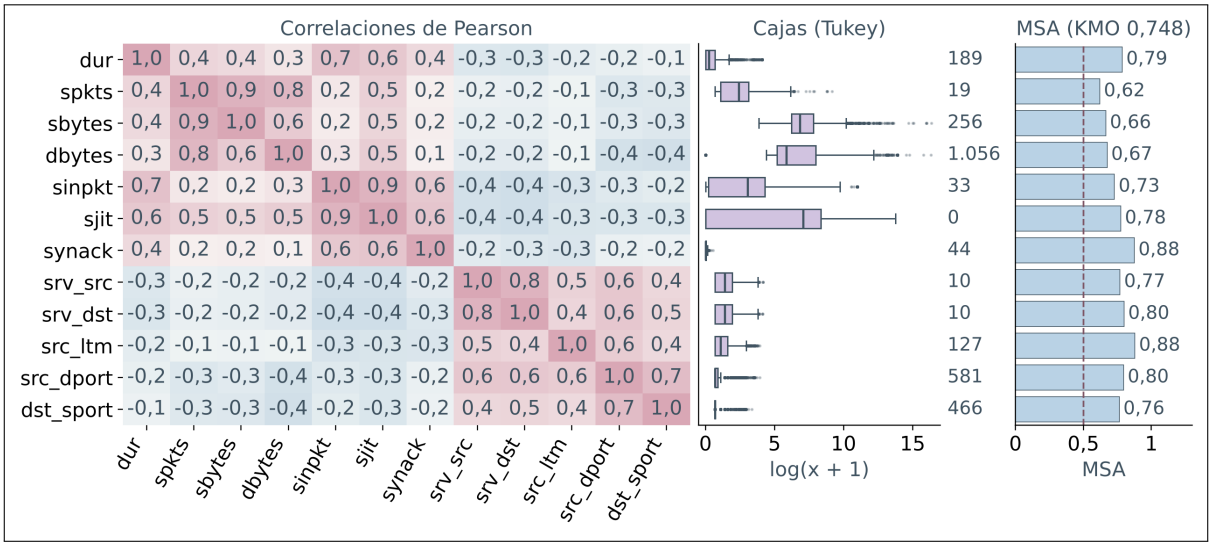


Figura 1. Izquierda: matriz de correlaciones de Pearson de las 12 variables de X_num (mapa de calor; rosa positivo, azul negativo). Centro: diagramas de caja en escala $\log(x+1)$ con los límites de Tukey y, a la derecha, la cantidad de flujos atípicos. Derecha: MSA por variable (línea punteada: umbral 0,50) y KMO global. Fuente: elaboración propia con UNSW-NB15.

Descriptiva y supuestos. La matriz de la Figura 1 muestra tres zonas: volumen (`spkts`–

sbytes 0,86), temporización (sinpkt–sjit 0,85) y repetición (ct_srv_src–ct_srv_dst 0,85), esta última con correlaciones negativas con el resto ($-0{,}13$ a $-0{,}43$); sjit también se asocia con el volumen (0,46 a 0,55), así que los bloques no están aislados. Los histogramas (Figura 2) son multimodales y con muchos ceros en volumen y temporización, y con cola derecha en repetición (los de las 12 variables están en resultados/figuras). Los supuestos se cumplen: determinante $6{,}3 \times 10^{-5}$ (mayor que 10^{-5}), KMO global 0,748 (Kaiser, 1974), MSA mínima 0,621 (spkts) y Bartlett significativo (χ^2 = 48.303,5; 66 gl; p <0,001). Con 14 candidatas el determinante era $1{,}0 \times 10^{-7}$ por la redundancia de dpkts (r = 0,95 con dbytes) y de rate (r = $-0,91$ con sinpkt); quitar solo dpkts no bastaba ($5{,}2 \times 10^{-6}$), así que excluimos ambas. Las frecuencias de las categóricas están en la Tabla 6.

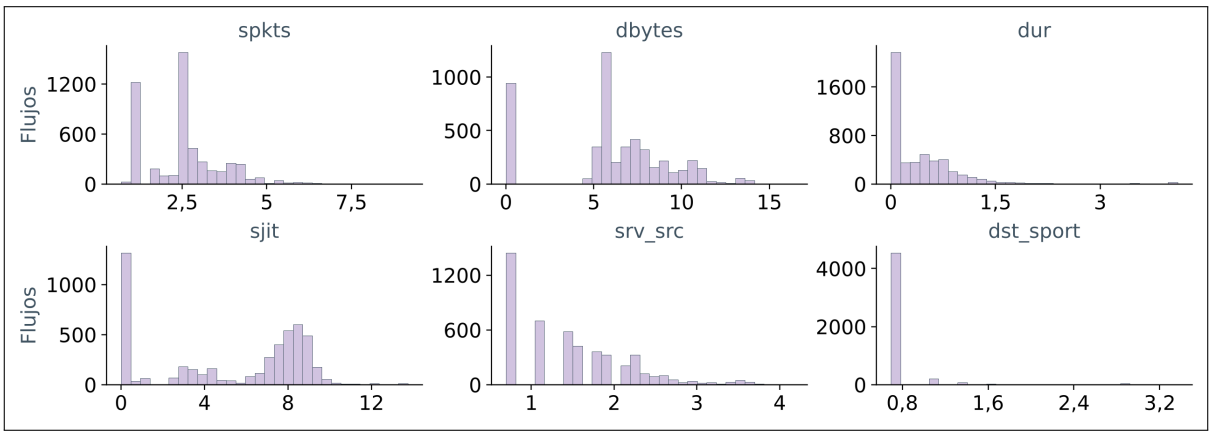


Figura 2. Histogramas de seis variables de X_num (dos por bloque) en escala $\log(x+1)$; las 12 están en resultados/figuras. Fuente: elaboración propia con UNSW-NB15.

3. Análisis de componentes principales

Estandarización y número de componentes. Aplicamos prcomp(X_num, scale. = TRUE). Las varianzas de las variables logarítmicas van de 0,002 (log_synack) a 13,61 (log_sjit), y tres de ellas reúnen el 83,2 % de la varianza total. Sin estandarizar, la primera componente captaría el 66,4 % de la variabilidad (43,3 % con estandarización) y reflejaría las escalas y no las relaciones; por eso usamos la matriz de correlaciones (Johnson and Wichern, 2007). Los autovalores son 5,195, 2,026 y 1,721 para las tres primeras y el cuarto cae a 0,759: solo tres superan 1 (Kaiser) y el codo (Figura 3, izquierda) se aplana tras la tercera. Explican 43,3, 16,9 y 14,3 % de la varianza (74,5 % acumulado) y cada una de las restantes menos del 6,4 %. Retenemos tres, interpretables y coincidentes en número con los bloques de H1.

Interpretación. Según la Tabla 2, en CP1 las siete variables de volumen y temporización tienen signo positivo (0,54 a 0,82) y las cinco de repetición, negativo ($-0{,}56$ a $-0{,}70$); aportan más log_sjit (13,1 %) y log_sinpkt (9,9 %). CP2 se construye con log_spkts (20,1 %), log_sbytes (15,7 %) y las repeticiones log_ct_srv_src y log_ct_srv_dst (12,7 y 11,8 %), todas en positivo. CP3, con log_sinpkt (20,1 %) y log_synack (19,6 %), en sentido opuesto a bytes y paquetes. Nombramos CP1 *magnitud del flujo frente a repetición*: opone el tamaño y el ritmo (paquetes, bytes, duración, espaciado y jitter) a la repetición de conexiones. CP2, *volumen de tráfico*, aunque no es un eje puro porque las repeticiones también cargan sobre ella (0,29 a 0,51). CP3, *ritmo lento*, por el espaciado entre paquetes y el tiempo del handshake.

Observaciones extremas. Medimos la distancia de cada flujo al centro en las tres

Tabla 2. Correlaciones entre cada variable y las tres componentes retenidas, en negrita las de $|r| \geq 0{,}5$, y calidad de representación acumulada (suma de cos2 en CP1–CP3).

Variable	Bloque esperado	CP1	CP2	CP3	cos2 total
`log_spkts`	Volumen	**0,64**	**0,64**	−0,35	0,94
`log_sbytes`	Volumen	**0,60**	**0,56**	−0,32	0,78
`log_dbytes`	Volumen	**0,64**	0,45	−0,39	0,76
`log_dur`	Temporización	**0,64**	0,27	0,40	0,64
`log_sinpkt`	Temporización	**0,72**	0,03	**0,59**	0,86
`log_sjit`	Temporización	**0,82**	0,23	0,36	0,87
`log_synack`	Temporización	**0,54**	0,00	**0,58**	0,63
`log_ct_srv_src`	Repetición	**−0,68**	**0,51**	0,08	0,73
`log_ct_srv_dst`	Repetición	**−0,70**	0,49	0,04	0,73
`log_ct_src_ltm`	Repetición	**−0,56**	0,45	0,00	0,52
`log_ct_src_dport_ltm`	Repetición	**−0,69**	0,43	0,41	0,82
`log_ct_dst_sport_ltm`	Repetición	**−0,61**	0,29	0,44	0,65

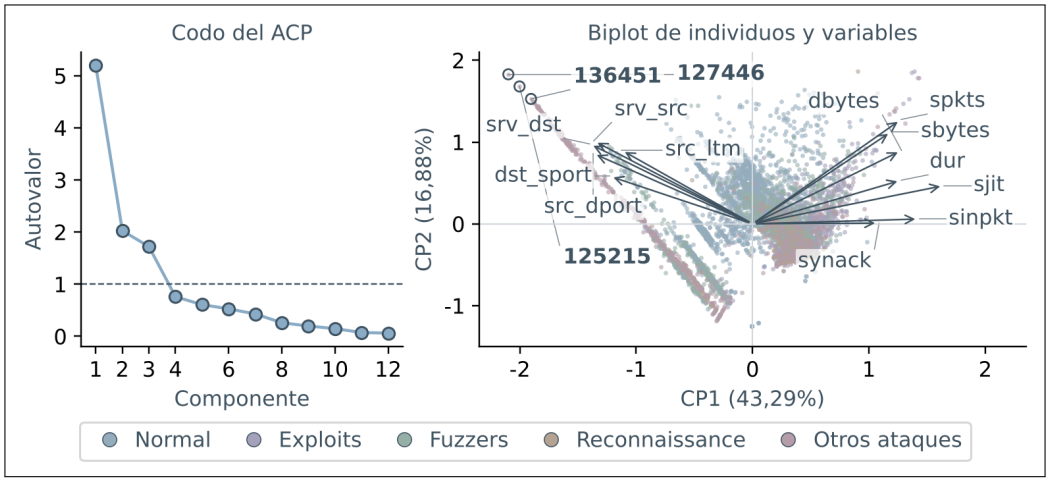


Figura 3. Gráfico del codo (izquierda) y biplot de CP1 y CP2 (derecha), con los flujos coloreados por `tipo_trafico`. Los tipos de tráfico se solapan en gran medida; los tres flujos extremos están rotulados con su identificador. Fuente: elaboración propia con UNSW-NB15.

componentes, ponderada por la varianza de cada una. Los tres más alejados (id 127446, 125215 y 136451; distancias 43,6, 38,0 y 32,6) tienen el mismo perfil: tráfico UDP al servicio DNS en estado INT, de 2 paquetes y 114 bytes, sin respuesta, de 6 a 9 microsegundos y del tipo “otros ataques” (Generic). Lo extremo es la repetición: `ct_srv_src` va de 45 a 50 (mediana 3) y `ct_src_ltm` de 33 a 50 (mediana 2), de ahí su puntaje muy negativo en CP1 ($-8{,}7$ a $-9{,}6$). No son errores: 160 flujos comparten ese patrón (159 son ataques, 151 Generic) y 129 forman parte del cluster 3 (sección 5), que tiene 147. En el biplot, el tipo de tráfico no separa grupos nítidos.

El ACP aísla la repetición frente al resto, pero volumen y temporización no se separan en CP1 (veredicto sobre H1 en la sección 8).

4. Análisis factorial

Los supuestos se cumplen (sección 2: determinante, Bartlett, KMO 0,748 y MSA mínimo 0,621), por lo que no se eliminó ninguna variable por MSA (`dpkts` y `rate` ya se habían excluido por el determinante).

Número de factores. Gráficamente, el análisis paralelo (Horn, 1965) de la Figura 4 deja tres autovalores muy por encima del azar; el cuarto (0,127; simulado 0,036) lo supera apenas,

porque con $n = 5.000$ se detecta casi cualquier estructura, y por eso sugiere cuatro. Analíticamente, el criterio de Kaiser sobre la matriz reducida da 4,93; 1,78; 1,44 y 0,43: tres factores, y el cuarto aportaría solo 4,8 % de la varianza común. Con cuatro, el bloque de repetición se parte en dos factores que correlacionan 0,62 (oblimin): sobreextracción. Retenemos **tres factores**, igual que el ACP.

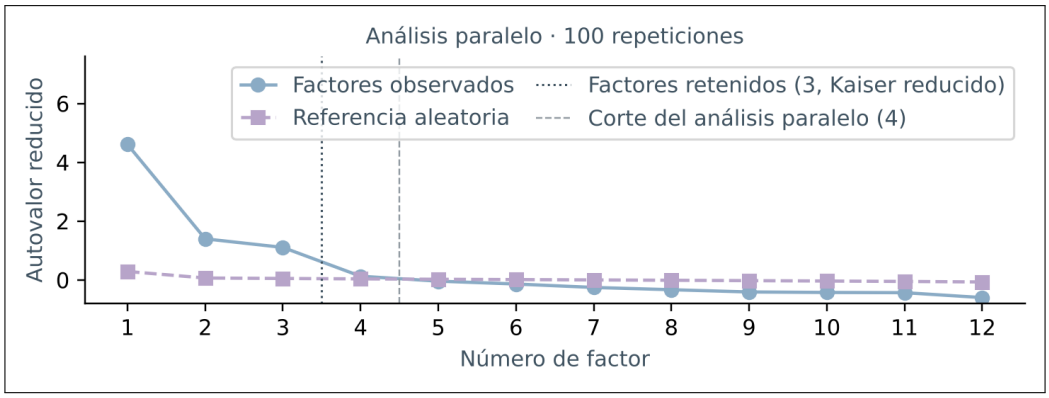


Figura 4. Análisis paralelo: tres autovalores quedan muy por encima de la referencia aleatoria y el cuarto (0,127) apenas la supera. Retenemos tres (Kaiser reducido). Fuente: elaboración propia con UNSW-NB15.

Estimador y caso Heywood. Con tres factores `log_spkts` alcanza una comunalidad mayor que 1 con minres (1,025) y ejes principales (1,095), y casi 1 con máxima verosimilitud (0,995); con cuatro factores también (minres 1,029, GLS 1,011). La causa es que `log_spkts` se reproduce casi por completo con el factor Volumen ($r = 0{,}86$ con `log_sbytes`), que tiene solo tres indicadores. La única combinación sin solución impropia es **tres factores con GLS** (comunalidad máxima 0,897), que además no obliga a quitar una variable; usamos GLS con rotación varimax (diagnóstico por estimador en `resultados/tablas`).

Interpretación. Según la Tabla 3, cada variable carga en un único factor (mínimo 0,54) y la menor diferencia con la segunda carga es 0,29 (`log_ct_src_ltm`), mayor que 0,20: no hay cargas cruzadas ambiguas. Nombramos **F1 Repetición de conexiones**, **F2 Temporización** y **F3 Volumen**, que coinciden con los bloques de H1 y explican 68,6 % de la varianza total. Los puntajes factoriales, que son estimaciones, quedan guardados.

Tabla 3. Matriz de patrón del AF (GLS, varimax; cargas $< 0{,}40$ ocultas con "–"), comunalidades (h^2) y unicidades (u^2). Fuente: elaboración propia con UNSW-NB15.

Variable	F1 Repetición	F2 Temporización	F3 Volumen	h^2	u^2
`log_ct_src_dport_ltm`	0,91	–	–	0,90	0,10
`log_ct_srv_src`	0,77	–	–	0,68	0,32
`log_ct_srv_dst`	0,76	–	–	0,69	0,31
`log_ct_dst_sport_ltm`	0,70	–	–	0,59	0,41
`log_ct_src_ltm`	0,54	–	–	0,36	0,64
`log_sinpkt`	–	0,87	–	0,83	0,17
`log_sjit`	–	0,81	–	0,87	0,13
`log_synack`	–	0,70	–	0,51	0,49
`log_dur`	–	0,68	–	0,56	0,44
`log_spkts`	–	–	0,91	0,87	0,13
`log_sbytes`	–	–	0,81	0,70	0,30
`log_dbytes`	–	–	0,79	0,70	0,30
% de la varianza total	25,1	22,4	21,1	Total 68,6	

Ajuste global. Es flojo: RMSR 0,051, TLI 0,609 y RMSEA 0,239, y el ajuste exacto se rechaza ($\chi^2 = 9.452$, 33 g.l., $p < 0{,}001$; esperable con $n = 5.000$). Con cuatro factores mejora (TLI 0,79) pero con solución impropia: es el costo de la parsimonia, y usamos el AF de forma descriptiva (Hair et al., 2019; Revelle, 2024).

Tabla 4. Comparación entre el ACP (Etapa 1) y el AF (Etapa 2) sobre las mismas 12 variables. Fuente: elaboración propia con UNSW-NB15.

	ACP	AF (GLS, varimax)
Varianza que usa	Total (diagonal de R igual a 1)	Solo la común (la unicidad se separa)
Retenido	3 componentes, 74,5 % de la varianza total	3 factores, 68,6 % de la varianza total
Qué representan los ejes	Combinaciones lineales que resumen la varianza; ejes mixtos	Causas latentes de las correlaciones; bloques
Variables en cada eje	CP1: volumen y temporización (+0,54 a +0,82) frente a repetición ($-0{,}56$ a $-0{,}70$); CP2: volumen (+0,45 a +0,64) y repetición (+0,29 a +0,51); CP3: `sinpkt`, `synack`, `ct_*_port` (+0,41 a +0,59) frente a volumen ($-0{,}32$ a $-0{,}39$)	F1: las cinco `ct_*`; F2: `dur`, `sinpkt`, `sjit`, `synack`; F3: `spkts`, `sbytes`, `dbytes`

ACP versus AF. La Tabla 4 compara ambas técnicas. Para esta base y esta pregunta (qué bloques de variables se mueven juntos, H1) resulta más adecuado el **AF**: sus tres ejes son los tres bloques esperados, mientras que cada componente del ACP combina tiempo, volumen y repetición y obliga a leer contrastes. El ACP conviene para resumir (74,5 % frente a 68,6 %), con puntajes exactos y sin soluciones impropias, por lo que lo usamos como predictor en la regresión logística. La ventaja del AF es de interpretación, no de ajuste. Respecto de H1, el AF recupera los tres bloques con cargas $\geq 0{,}54$ y coincide exactamente con el cluster de variables (sección 5).

5. Análisis de conglomerados

Agrupamos los 5.000 flujos con las 12 variables de X_num estandarizadas, con distancia euclidiana y Ward.D2 (el dendrograma de los flujos, cortado en seis grupos, está en `resultados/figuras/11_cluster_dendrograma.png`; el de las variables, en la Figura 6), y comparamos con k-means (`nstart` = 25, semilla 42). El codo y el silhouette promedio para k = 2 a 8 están en la Figura 5.

La elección de k = 6 es defendible pero no contundente. El silhouette de k-means es 0,328 (k = 2), 0,302 (3), 0,327 (4), 0,329 (5), 0,333 (6), 0,277 (7) y 0,284 (8): k = 2, 4, 5 y 6 empatan en la práctica (diferencias menores que 0,007), todos están bajo 0,5 (estructura débil; Kaufman and Rousseeuw, 1990) y lo claro es la caída en k = 7. El codo más marcado está en k = 4. Elegimos k = 6 por el silhouette máximo y porque aísla al grupo de 147 flujos (cluster 3), diluido con k = 4 en uno de 215. Los clusters 2 y 6 están mal separados: su silhouette medio es 0,063 y 0,119, con 35,6 y 26,8 % de flujos de silhouette negativo.

Jerárquico, k-means y preprocesamiento. Los seis grupos de Ward se corresponden casi uno a uno con los de k-means: la tabla cruzada tiene 4.508 de los 5.000 flujos en la correspondencia principal (90,2 %; ARI = 0,813), y los 147 flujos del cluster 3 de k-means

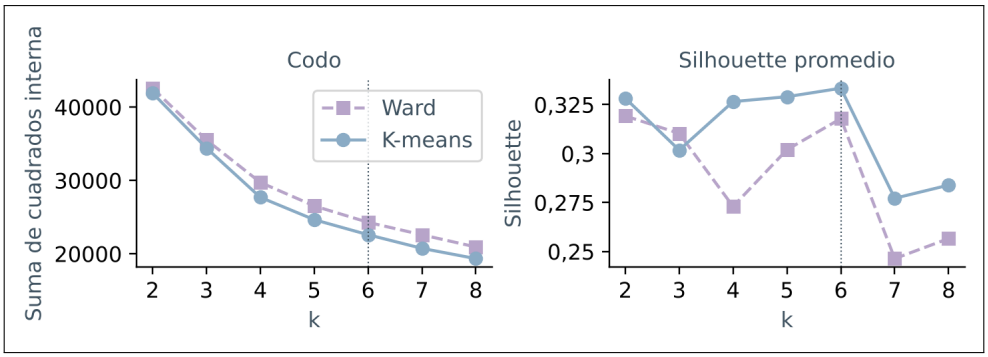


Figura 5. Codo (izquierda) y silhouette promedio (derecha) para k = 2 a 8, con Ward y k-means. El silhouette de k-means es máximo en k = 6 (0,333), casi igual a k = 2, 4 y 5, y cae en k = 7. Fuente: elaboración propia con UNSW-NB15.

casi íntegros en un grupo de Ward (143). Perfilamos k-means. Sobre los puntajes de las tres componentes del ACP el ARI baja a 0,708 (0,605 con Ward): los grupos cambian. Los clusters 3 (147 flujos) y 5 (1.126 de 1.135) se conservan casi íntegros, el 4 se divide en dos (1.578 y 515 de 2.131) y el 6 se reparte entre cuatro grupos (95, 42, 71 y 76 flujos). Es esperable: las componentes descartan 25,5 % de la varianza y, sin reescalar, la primera pesa más en la distancia.

Cluster de variables. Ward sobre las columnas estandarizadas (Figura 6, derecha) separa tres bloques: temporización, volumen y repetición de conexiones (las cinco `ct_*`). Coincide exactamente con los tres factores del AF (ARI = 1,000) y con H1; las componentes del ACP mezclan bloques.

Perfiles. La Figura 6 (izquierda) da las medias estandarizadas por cluster y la Tabla 5, los nombres, rasgos en unidades originales y el cruce con `tipo_trafico` (descriptivo; no se usó para agrupar).

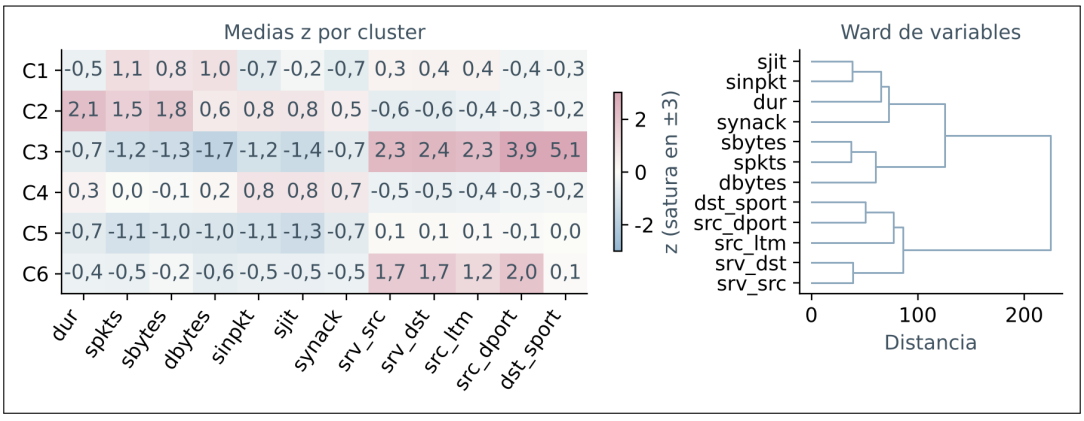


Figura 6. Izquierda: medias estandarizadas de cada variable por cluster de k-means (rojo: por encima del promedio; azul: por debajo). Derecha: dendrograma Ward de las 12 variables, que separa tres bloques. Fuente: elaboración propia con UNSW-NB15.

Hipotetizamos (sin poder contrastarlo) que el cluster 1 es transferencia legítima de datos; el 3, ráfagas automáticas de paquetes pequeños sin respuesta (130 de 147 son Generic); el 5, consultas breves, y el 6, conexiones repetidas a un mismo servicio. Dada la silhouette baja, los leemos como descripción útil y no como categorías naturales. Los clusters se guardaron en la base para el ACM.

Tabla 5. Perfil de los seis clusters (k-means, k = 6): tamaño, porcentaje de ataques, tipos de tráfico más frecuentes (%) y rasgos distintivos (promedios originales). Fuente: elaboración propia con UNSW-NB15.

Cluster	n (%)	% ataq.	Tipos de tráfico (%)	Rasgos distintivos
1 Transferencias voluminosas de respuesta rápida	898 (18,0)	1,4	Normal 98,6	54 paquetes y 48.466 bytes de respuesta; intervalos cortos
2 Sesiones largas y pesadas	405 (8,1)	72,6	Exploits 44,2; Normal 27,4; Fuzzers 19,3	Duración 10,4 s; 183.786 bytes enviados; poca repetición
3 Ráfagas repetidas sin respuesta	147 (2,9)	100	Otros ataques 96,6	2 paquetes, 0 bytes de respuesta, duración casi nula; `ct_dst_sport_ltm` 12,0 ($+5{,}1$ desvíos)
4 Conexiones medianas de ritmo lento	2.131 (42,6)	71,0	Exploits 30,9; Normal 29,0; Fuzzers 19,5; Recon. 12,0	Duración 1,03 s; 11,9 paquetes; `sinpkt` 729 ms; `sjit` y `synack` altos
5 Intercambios mínimos	1.135 (22,7)	46,3	Normal 53,7; Fuzzers 16,3; Otros ataques 16,1	Duración 0,006 s; 2,4 paquetes; volumen bajo; repetición promedio
6 Repetición moderada de bajo volumen	284 (5,7)	34,9	Normal 65,1; Fuzzers 21,8	`ct_srv_*` $\approx$ 17; `ct_src_dport_ltm` 5,9 ($+2{,}0$ desvíos); volumen bajo

6. Análisis de correspondencias simple y múltiples

6.1. Construcción de X_cat

X_cat tiene siete variables (Tabla 6). `tipo_trafico` entra como variable activa, a diferencia del ACP, el AF y los conglomerados, porque se piden asociaciones entre categóricas, incluida la de contexto; son descriptivas y en parte esperables. Todas las categorías superan el 5 % (la menor, 6,98 %), por lo que no fusionamos ninguna. Para `dur` y `sbytes`, muy asimétricas (`sbytes`: mediana 918 y media 16.918 bytes), usamos terciles: cortes de igual amplitud dejarían casi todos los flujos en una categoría, y los terciles dan 33 % por categoría sin usar la etiqueta. `respuesta_destino` separa los flujos con `dpkts` = 0 ("sin respuesta"), un corte natural.

Tabla 6. X_cat: variables, origen y categorías finales (% de los 5.000 flujos). Fuente: elaboración propia con UNSW-NB15.

Variable	Origen	Categorías (%)
`tipo_trafico`	`attack_cat` agrupada	Normal 48,2 · Exploits 18,4 · Fuzzers 15,0 · Otros ataques 11,5 · Reconnaissance 7,0
`proto_g`	`proto` agrupada	tcp 70,8 · udp 21,9 · otros 7,3
`service_g`	`service` agrupada	ninguno 60,0 · http 17,4 · otros 12,3 · dns 10,3
`state_g`	`state` agrupada	FIN 69,3 · INT 18,3 · CON_otros 12,4
`respuesta_destino`	`dpkts` = 0 o >0	con_respuesta 81,1 · sin_respuesta 18,9
`duracion_banda`	terciles de `dur` (cortes 0,0185 s y 0,709 s)	corta 33,3 · media 33,3 · larga 33,3
`bytes_origen_banda`	terciles de `sbytes` (cortes 564 y 1.580 bytes)	bajos 33,8 · medios 33,6 · altos 32,6

6.2. Análisis de correspondencias simple: `tipo_trafico` × `state_g`

Elegimos `state_g` por encarnar H3 y ser la más asociada al tipo de tráfico (V de Cramér 0,405, frente a 0,320 de `service_g` y 0,302 de `proto_g`). La asociación es significativa ($\chi^2 = 1.637{,}2$, 8 gl, $p < 0{,}001$) y dos dimensiones recogen el 100 % de la inercia (77,9 y 22,1 %). La Dim 1 opone INT ($-1{,}009$) a CON_otros ($+0{,}662$): INT concentra el 62,3 % de Otros ataques, el 33,0 % de Fuzzers y el 26,1 % de Reconnaissance contra el 5,9 % de Normal, y CON_otros al 24,0 % de Normal y no más del 3,8 % de cada ataque. Exploits no sigue el patrón (90,4 % en

FIN; Normal, 70,0 %). (mapa en `13_acs_mapa.png`).

6.3. Análisis de correspondencias múltiples

Con $Q = 7$ y $J = 23$, la inercia total cruda es $J/Q - 1 = 2{,}286$ y el autovalor medio $1/Q = 0{,}143$ (seis lo superan: criterio poco exigente). Los porcentajes crudos son bajos (Dim 1: 27,4 %, autovalor 0,627; Dim 2: 14,0 %, 0,321; acumulado 41,4 %). `mjca` del paquete `ca` (Nenadić and Greenacre, 2007) usa por defecto la corrección de Greenacre (`lambda = "adjusted"`; Greenacre, 2017), que reportamos: inercia total ajustada 0,414, Dim 1 = 77,0 %, Dim 2 = 10,4 %, Dim 3 = 1,8 % (gráfico de inercia en `14_acm_inercia.png`); el plano suma 87,4 % de inercia ajustada, cifra optimista que debe leerse junto con el 41,4 % crudo. Retenemos **dos dimensiones**: el codo de los autovalores se marca tras la Dim 2 y la tercera aporta menos del 2 % ajustado. Contribuciones, cos2 y η^2 salen de `FactoMineR::MCA` (Lê et al., 2008; Husson et al., 2017).

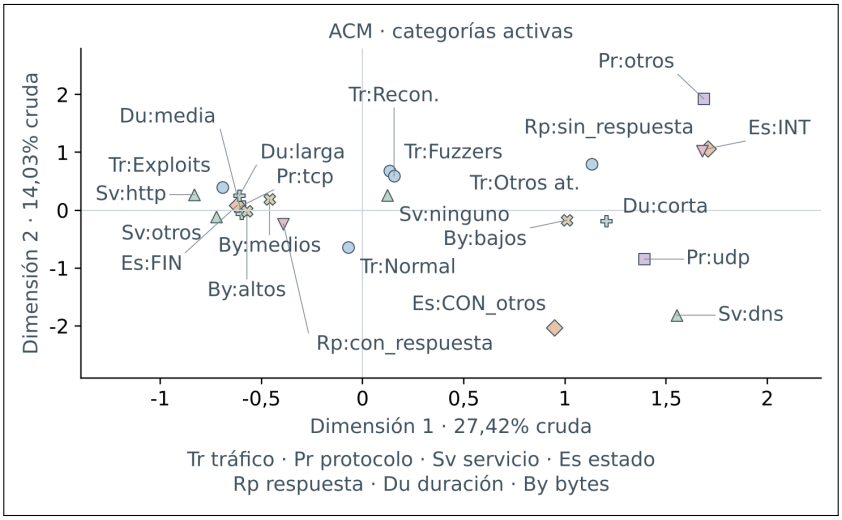


Figura 7. Mapa de categorías del ACM (Dim 1 y Dim 2, porcentajes crudos). A la derecha, flujos breves, sin respuesta y no TCP; a la izquierda, TCP, FIN, mayor duración y volumen. Fuente: elaboración propia con UNSW-NB15.

Tabla 7. Categorías que más contribuyen (Ctr, %) a las dimensiones 1 y 2, con su cos2 en cada una. Fuente: elaboración propia con UNSW-NB15.

Categoría	Ctr 1	cos2 1	Ctr 2	cos2 2	Categoría	Ctr 1	cos2 1	Ctr 2	cos2 2
`state_g`: INT	12,1	0,651	9,2	0,252	`state_g`: FIN	6,1	0,866	0,2	0,016
`respuesta`: sin	12,1	0,656	8,7	0,242	`proto_g`: tcp	5,9	0,886	0,1	0,010
`duracion`: corta	11,0	0,726	0,5	0,018	`service_g`: dns	5,7	0,277	15,2	0,381
`proto_g`: udp	9,7	0,545	6,9	0,199	`state_g`: CON_otros	2,5	0,127	22,8	0,585
`bytes_origen`: bajos	7,8	0,520	0,4	0,015	`proto_g`: otros	4,7	0,223	12,0	0,291

Dim 1: "conexión efímera sin respuesta" frente a "sesión TCP completa". La construyen INT, sin_respuesta, duración corta, UDP y bytes bajos contra FIN y TCP (Tabla 7); discriminan sobre todo `state_g` ($\eta^2 = 0{,}909$), `proto_g` (0,891) y `duracion_banda` (0,726), y `tipo_trafico` apenas (0,242; Figura 8, derecha). **Dim 2: "DNS/UDP con estado CON" frente a "protocolo *otros* no establecido".** La construyen CON_otros (22,8 %), dns (15,2 %) y proto *otros* (12,0 %), y Normal queda del lado DNS/CON (cos2 0,384); los demás tipos tienen cos2 de 0,03 a 0,08 (posición solo indicativa). **Redundancia:** `proto_g` = *otros* implica `service_g` = ninguno (363 de 363) y casi siempre INT (344 de 363); FIN solo ocurre en TCP; todo flujo INT

es sin_respuesta (913 de 913) y casi todo flujo sin respuesta es INT (913 de 945). Por eso se agrupan en el mapa.

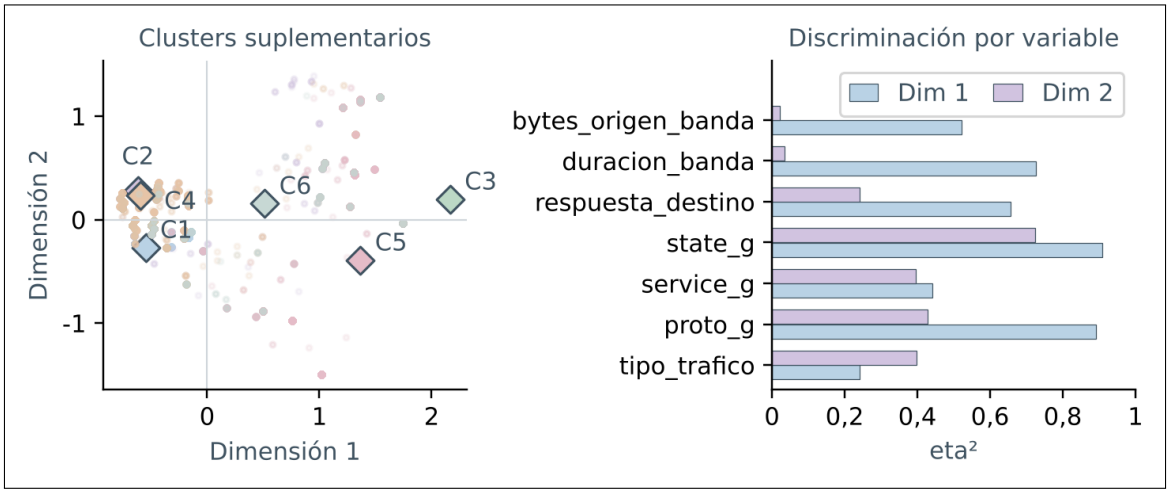


Figura 8. Izquierda: clusters de la Etapa 3 proyectados como suplementarios (rombos) sobre las observaciones; los clusters 1, 2 y 4 se superponen (Dim 1 entre $-0{,}62$ y $-0{,}55$). Derecha: η^2 por variable en Dim 1 y Dim 2; `state_g` y `proto_g` dominan la Dim 1. Fuente: elaboración propia con UNSW-NB15.

Clusters suplementarios (Figura 8). No ocupan zonas totalmente distintas: los clusters 1, 2 y 4 se superponen en la zona TCP/FIN y se distinguen por duración y bytes. El cluster 3 (Dim 1 = 2,16) queda aislado a la derecha: 100 % INT, sin respuesta, duración corta y bytes bajos, y 87,8 % UDP y dns. El 5 (Dim 1 = 1,37) es breve (96,1 % corta), UDP (73,9 %) y con 45,6 % CON_otros; el 6 es intermedio. Los cos2 en el plano son bajos salvo el 5 (0,59): lectura parcial.

Tres asociaciones. (i) *Otros ataques con INT, sin respuesta y flujos pequeños*: 62,3 % INT y 63,2 % sin respuesta (Normal: 5,9 y 6,9 %), 62,8 % con bytes bajos. (ii) *Normal con CON_otros y con respuesta* (24,0 % y 93,1 %); el DNS no es exclusivo de lo normal (14,4 % frente a 27,6 % en Otros ataques). (iii) *Exploits con TCP/FIN, duración larga y http*: 54,7 % larga (Normal: 23,7 %) y 44,2 % http (Normal: 9,9 %). Son frecuencias de laboratorio; el ACM es descriptivo.

7. Regresión logística

Diseño. Modelamos la probabilidad de que un flujo sea ataque (`label` = 1) con regresión logística (Hosmer et al., 2013) para contrastar H4. Partimos la muestra 70/30 en 3.501 flujos de entrenamiento y 1.499 de prueba (51,8 % de ataques en ambos), estratificando por `attack_cat` (semilla 42; partición en `muestra_particion_70_30.xlsx`); el testing set original no está en el repositorio. Reajustamos el ACP solo con el entrenamiento para evitar fuga (cargas casi idénticas a las de la sección 3). Los predictores son CP1 a CP3 y `proto_g` (referencia tcp), `service_g` (ninguno) y `state_g` (FIN). Excluimos TTL (separación perfecta artefactual), `tipo_trafico` y `attack_cat` (fuga). M1 usa solo los puntajes (AIC 3.734,2; R^2 de McFadden 0,232; AUC en prueba 0,795) y M2 agrega protocolo, servicio y estado (AIC 2.876,5; R^2 0,411; AUC 0,882); la razón de verosimilitud favorece a M2 ($\chi^2 = 871{,}7$; 7 gl; $p < 0{,}001$), que elegimos.

Odds ratios. Con lo demás fijo, un desvío más de CP1 (magnitud del flujo frente a repetición) multiplica las chances de ataque por 5,01 (IC 95 % por verosimilitud perfilada: 4,11 a 6,15), uno más de CP3 (ritmo lento) por 2,54 (2,25 a 2,86) y uno más de CP2 (volumen de

tráfico) las reduce un 41 % (OR 0,59; 0,53 a 0,67). Los OR de las categóricas son condicionales y la colinealidad es alta (VIF 18,4 en `udp` y 17,1 en INT; las componentes llegan a 3,64), por lo que se interpretan solo en su dirección: INT frente a FIN (OR 375,1; IC 107,0 a 1.317,7) y dns frente a ninguno (19,1; 9,0 a 42,5), más chances de ataque; CON_otros frente a FIN (0,19; 0,08 a 0,44), menos. La inestabilidad lo confirma: al rehacer la partición el OR de INT pasó de 93 a 375 y el de udp de 0,42 a 0,14. Así, dns tiene menos ataques que servicio ninguno (32 frente a 49 %) pero OR ajustado 19,1. Los demás OR están en `resultados/tablas`.

Desempeño en prueba. Con umbral 0,5, M2 logra AUC = 0,882 (IC 95 % de DeLong con `pROC` (Robin et al., 2011): 0,864 – 0,900), exactitud 0,818, sensibilidad 0,915, especificidad 0,713 y precisión 0,775. Domina el falso positivo: 207 normales marcados como ataque (28,7 %) frente a 66 ataques no detectados (8,5 %; Figura 9), un costo aceptable al detectar intrusiones.

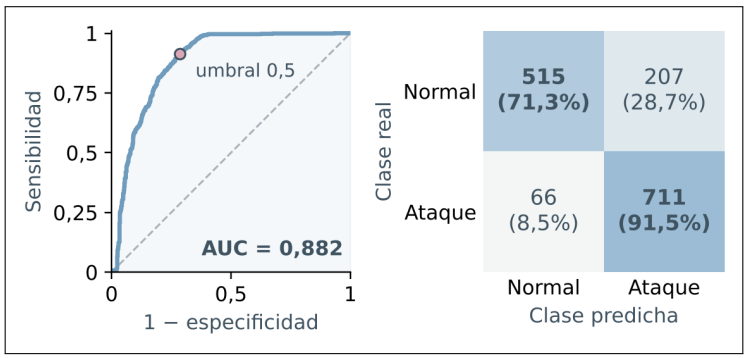


Figura 9. Curva ROC y matriz de confusión de M2 en la prueba (umbral 0,5). La diagonal punteada corresponde al azar (AUC = 0,5). Fuente: elaboración propia con UNSW-NB15.

Diagnóstico. No hay separación (convergió sin avisos). Según la distancia de Cook, 166 flujos (4,7 %) superan $4/n$, pero el máximo es 0,019. La calibración por deciles es buena entre el 5 y el 9 (diferencias de hasta 2,6 puntos) e irregular en los extremos: los tres deciles bajos sobreestiman en proporción (0,02, 0,06 y 0,15 contra 0,007, 0,013 y 0,020 observadas) y el 10 sobreestima unos 12 puntos (0,96 contra 0,84).

Cierre. H4 se confirma: el AUC en prueba es 0,882 y el límite inferior de su IC (0,864) también supera 0,80; con las componentes solas (M1, 0,795) no se cumpliría. La probabilidad es relativa a esta base (52 % de ataques, sin duplicados, ataques sintéticos): interpretamos odds ratios, no prevalencias.

8. Integración y conclusiones

La Tabla 8 integra las técnicas, una fila por técnica. Los veredictos sobre las hipótesis son:

- **H1, confirmada.** El AF y el cluster de variables dan los mismos tres bloques (ARI = 1,000); el ACP también pide tres componentes, pero no los recupera como ejes puros.
- **H2, parcialmente confirmada.** El cluster 1 es casi todo normal y el 3 todo ataque, pero los otros cuatro mezclan ambos; además, el cluster 3 distingue un tipo (Generic), lo que contradice la segunda parte de la hipótesis.
- **H3, parcialmente confirmada.** INT y sin respuesta se asocian con Otros ataques, pero Exploits es casi todo FIN con respuesta, y dns y http no son exclusivos de lo normal.
- **H4, confirmada.** El AUC de prueba (0,882) y el límite inferior de su IC superan 0,80; solo

Tabla 8. Integración de las técnicas: estructura hallada, protagonistas y relación con las demás. Fuente: elaboración propia con UNSW-NB15.

Técnica	Estructura	Protagonistas	Coincidencias y discrepancias
ACP (§3)	3 componentes (74,5 %): CP1 magnitud frente a repetición, CP2 volumen de tráfico, CP3 ritmo lento	CP1: `sjit`, `sinpkt` frente a las cinco `ct_*`; CP2: `spkts`, `sbytes`	Aísla la repetición, como el AF; sus ejes son mixtos
AF (§4)	3 factores (GLS, varimax), 68,6 %: Repetición, Temporización, Volumen	Cargas de 0,54 a 0,91, sin cruces ambiguos	Idéntico al cluster de variables (ARI = 1,000); ajuste global flojo (TLI 0,609)
Cluster de variables (§5)	3 bloques (Ward): temporización, volumen, repetición	Las cinco `ct_*` juntas; `sinpkt`–`sjit` (0,85)	Igual al AF; el ACP mezcla los bloques
Cluster de observaciones (§5)	6 grupos (k-means), silhouette 0,333, estructura débil	C1 transferencias (1,4 % ataque); C3 ráfagas sin respuesta (100 %); C2 y C6 mal separados	Se ordenan en CP1 y F1; en el ACM, C3 queda aislado en Dim 1; con puntajes del ACP, ARI = 0,708
ACS y ACM (§6)	ACS: tipo $\times$ estado (V = 0,405). ACM: 2 dim., 41,4 % crudo (87,4 % ajustado)	Dim 1: INT, sin respuesta, UDP frente a FIN y TCP; Dim 2: DNS/CON frente a protocolo *otros*	Dim 1 reproduce CP1 y el cluster 3; `tipo_trafico` pesa poco en Dim 1 (η^2 = 0,242)
Logística (§7)	M2: AUC 0,882 en prueba (IC 95 % 0,864–0,900)	CP1 (OR 5,01), CP3 (2,54), CP2 (0,59); INT, CON_otros (0,19), dns	Coincide con el ACM en INT y CON_otros; los OR categóricos son inestables

con las componentes (M1, 0,795) no se cumpliría.

Bloques y clusters. Los clusters se ordenan sobre todo por CP1 ($\eta^2 = 0{,}86$; CP2 y CP3, 0,65) y por los factores (F2 Temporización 0,78, F3 Volumen 0,73, F1 Repetición 0,72): el cluster 3 tiene F1 = $+3{,}78$ (repetición) y el 2, F2 = $+0{,}88$ (sesiones largas). Es en parte circular, pero el ACM, que no intervino, lo confirma: el cluster 3 es todo INT y sin respuesta, y los clusters 1, 2 y 4 se superponen en TCP/FIN.

Sorpresas. (i) La repetición alta no distingue ataques: el cluster 3 es todo ataque y el 6, un tercio. (ii) El cluster 3 aísla Generic. (iii) El caso Heywood de `log_spkts`. (iv) La inestabilidad de los OR categóricos.

Limitaciones. Los ataques son sintéticos y, sin duplicados, las proporciones no son prevalencias: la probabilidad logística es relativa a esta base (52 % de ataques); excluimos TTL por artefacto; los `ct_*` usan ventanas de 100 conexiones, así que los flujos no son independientes; k = 6 empata en silhouette con k = 2, 4 y 5; los terciles del ACM son un criterio estadístico; conservamos los atípicos (34,3 % de los flujos tiene al menos uno), que pueden influir en componentes y factores; usamos una sola partición 70/30; y los datos son observacionales: describimos asociaciones, no causas.

Próximos pasos. Procesar el testing set oficial, validar de forma cruzada, regularizar para estabilizar los OR y recalibrar el umbral según el costo de los errores.

Conclusión. Los flujos se organizan en tres patrones, volumen, temporización y repetición, que junto con el estado, el servicio y el protocolo permiten estimar la probabilidad de ataque (AUC 0,882 en prueba), aunque con 28,7 % de los normales marcados como ataque. A nuestro juicio (valoración del grupo), lo más sólido es la estructura de variables, que tres técnicas recuperan, y lo más frágil, la lectura de los grupos y de los odds ratios aislados; el estado de la conexión parece aportar más que cualquier patrón numérico.

Referencias bibliográficas

Greenacre, M. (2017). *Correspondence Analysis in Practice*. Chapman and Hall/CRC, Boca Raton, FL, 3 edition.

Hair, J. F., Black, W. C., Babin, B. J., and Anderson, R. E. (2019). *Multivariate Data Analysis*. Cengage, Andover, Hampshire, 8 edition.

Horn, J. L. (1965). A rationale and test for the number of factors in factor analysis. *Psychometrika*, 30(2):179–185.

Hosmer, D. W., Lemeshow, S., and Sturdivant, R. X. (2013). *Applied Logistic Regression*. Wiley, Hoboken, NJ, 3 edition.

Husson, F., Lê, S., and Pagès, J. (2017). *Exploratory Multivariate Analysis by Example Using R*. Chapman and Hall/CRC, Boca Raton, FL, 2 edition.

Johnson, R. A. and Wichern, D. W. (2007). *Applied Multivariate Statistical Analysis*. Pearson, 6 edition.

Kaiser, H. F. (1974). An index of factorial simplicity. *Psychometrika*, 39(1):31–36.

Kaufman, L. and Rousseeuw, P. J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley, New York.

Lê, S., Josse, J., and Husson, F. (2008). FactoMineR: An R package for multivariate analysis. *Journal of Statistical Software*, 25(1):1–18.

Moustafa, N. and Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. In *Military Communications and Information Systems Conference (MilCIS)*. IEEE.

Moustafa, N. and Slay, J. (2016). The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Information Security Journal: A Global Perspective*, 25(1–3):18–31.

Nenadić, O. and Greenacre, M. (2007). Correspondence analysis in R, with two- and three-dimensional graphics: The ca package. *Journal of Statistical Software*, 20(3):1–13.

R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria.

Revelle, W. (2024). *psych: Procedures for Psychological, Psychometric, and Personality Research*. Northwestern University, Evanston, Illinois. Paquete de R.

Robin, X., Turck, N., Hainard, A., Tiberti, N., Lisacek, F., Sanchez, J.-C., and Müller, M. (2011). pROC: an open-source package for R and S+ to analyze and compare ROC curves. *BMC Bioinformatics*, 12:77.

A. Anexo: código fuente

Los scripts se ejecutan desde la carpeta principal del repositorio. Primero se corre `R/ejecutar_todo.R`, que genera las tablas en `resultados/tablas`; luego los scripts de Python dibujan las figuras a partir de esas tablas. Todos los procesos aleatorios usan la semilla 42.

Scripts de R

`R/00_datos.R` · Etapa 0: importación, calidad, descriptiva y supuestos

```r
# ETAPA 0. Ejecutar desde la carpeta principal del proyecto.
# Instalar una sola vez: install.packages(c("readxl", "psych"))
# Ejecutar: source("R/00_datos.R", encoding = "UTF-8")
library(readxl)
library(psych)
set.seed(42)

# 1. Importar el mismo Excel que usamos en el script del diccionario.
ruta_excel <- "muestra_estratificada-simulado-de-ataques.xlsx"
base <- read_excel(path = ruta_excel, sheet = "muestra")
info <- read_excel(path = ruta_excel, sheet = "diccionario")
info <- info[match(names(base), info$variable), ]
stopifnot(!anyNA(info$variable), !anyDuplicated(info$variable))
base_original <- base

diccionario <- data.frame(
  nombre = info$variable,
  descripcion = info$descripcion,
  unidad = info$unidad,
  tipo = ifelse(info$tipo %in% c("continua", "conteo"), "numérica", "categórica")
)
print(dim(base))
print(table(diccionario$tipo))
if (interactive()) View(diccionario)

# Guardar solo las tablas necesarias, con nombres fáciles de identificar.
dir.create("resultados/tablas", recursive = TRUE, showWarnings = FALSE)
dir.create("resultados/figuras", recursive = TRUE, showWarnings = FALSE)
guardar_tabla <- function(tabla, nombre) {
  write.csv(tabla, paste0("resultados/tablas/", nombre, ".csv"),
            row.names = FALSE, fileEncoding = "UTF-8", na = "")
}
guardar_tabla(diccionario, "diccionario_variables")

# 2. Elegir las 12 log_ acordadas para ACP, AF y clustering.
variables_num <- c(
  "log_dur", "log_spkts", "log_sbytes", "log_dbytes", "log_sinpkt", "log_sjit",
  "log_synack", "log_ct_srv_src", "log_ct_srv_dst", "log_ct_src_ltm",
  "log_ct_src_dport_ltm", "log_ct_dst_sport_ltm"
)
X_num <- as.data.frame(base[variables_num])
stopifnot(all(is.finite(as.matrix(X_num))))

# 3. Revisar faltantes, duplicados y valores incompatibles con su definición.
faltantes <- data.frame(variable = names(base), NA_encontrados = colSums(is.na(base)))
faltantes$textos_vacios <- sapply(base, function(x) sum(trimws(as.character(x)) == "", na.rm = TRUE))
columnas_perfil <- setdiff(names(base)[1:45], "id")
columnas_numericas <- names(base)[sapply(base, is.numeric)]
conteos <- info$variable[info$tipo == "conteo"]
valores <- as.matrix(base[columnas_numericas])

calidad <- data.frame(
  control = c("Valores NA", "Textos vacíos", "Filas duplicadas", "ID repetidos",
    "Perfiles originales duplicados sin id", "Celdas negativas", "Celdas no finitas sin NA",
    "Conteos no enteros", "TTL fuera de 0 a 255", "Ventana TCP fuera de 0 a 65535",
    "dur = 0 con spkts > 1 (revisar precisión)", "sbytes < spkts",
    "Indicadores usa_http/is_sm_ips_ports fuera de 0/1", "is_ftp_login fuera de 0/1",
    "label fuera de normal/ataque", "label y attack_cat inconsistentes"),
  cantidad = c(sum(is.na(base)), sum(faltantes$textos_vacios), sum(duplicated(base)),
    sum(duplicated(base$id)), sum(duplicated(base[columnas_perfil])),
    sum(valores < 0, na.rm = TRUE), sum(!is.finite(valores) & !is.na(valores)),
```

```r
    sum(sapply(base[conteos], function(x) sum(abs(x - round(x)) > 1e-8, na.rm = TRUE))),
    sum(base$sttl < 0 | base$sttl > 255 | base$dttl < 0 | base$dttl > 255, na.rm = TRUE),
    sum(base$swin < 0 | base$swin > 65535 | base$dwin < 0 | base$dwin > 65535, na.rm = TRUE),
    sum(base$dur == 0 & base$spkts > 1), sum(base$sbytes < base$spkts),
    sum(!base$usa_http %in% c(0, 1)) + sum(!base$is_sm_ips_ports %in% c(0, 1)),
    sum(!base$is_ftp_login %in% c(0, 1)), sum(!base$label %in% c("normal", "ataque")),
    sum((base$attack_cat == "Normal") != (base$label == "normal")))
)
ftp_revision <- base[!base$is_ftp_login %in% c(0, 1), c("id", "is_ftp_login", "ct_ftp_cmd")]
guardar_tabla(faltantes, "faltantes")
guardar_tabla(calidad, "calidad")
guardar_tabla(ftp_revision, "ftp_revision")
print(calidad)
print(ftp_revision)

# Atípicos: límites de Tukey sobre las 12 log_, sin borrar observaciones.
# fivenum usa bisagras de Tukey: pueden diferir de quantile cuando hay empates.
atipicos <- matrix(FALSE, nrow = nrow(base), ncol = length(variables_num))
limites <- data.frame()
for (i in seq_along(variables_num)) {
  variable <- variables_num[i]
  x <- X_num[[variable]]
  cinco <- fivenum(x)
  q1 <- cinco[2]
  q3 <- cinco[4]
  ric <- q3 - q1
  inferior <- q1 - 1.5 * ric
  superior <- q3 + 1.5 * ric
  atipicos[, i] <- x < inferior | x > superior
  limites <- rbind(limites, data.frame(variable = variable, q1_tukey = q1,
    mediana = cinco[3], q3_tukey = q3, limite_inferior = inferior, limite_superior = superior,
    bigote_inferior = min(x[x >= inferior & x <= superior]),
    bigote_superior = max(x[x >= inferior & x <= superior]), n_atipicos = sum(atipicos[, i])))
}
revision_atipicos <- data.frame(id = base$id, n_atipicos_original = base$n_atipicos,
                                n_atipicos_revision_12 = rowSums(atipicos))
guardar_tabla(limites, "atipicos_limites")
guardar_tabla(revision_atipicos, "atipicos_por_flujo")
cat("Flujos con atípicos en las 12 variables:", sum(rowSums(atipicos) > 0), "\n")

# Registrar qué se hace con cada problema; conservar la base recibida.
decisiones <- data.frame(
  tema = c("Faltantes", "Duplicados", "Valores incompatibles", "Código FTP",
           "Ceros y servicio ninguno", "Atípicos", "Indicador anterior"),
  decision = c("No imputar: no se encontraron faltantes en esta base.",
    "No eliminar filas: no se encontraron duplicados en esta base.",
    "Registrar los controles y revisar incidencias antes de cualquier corrección.",
    "Conservar el código 4 y el flujo; is_ftp_login no entra en X_num ni X_cat.",
    "Conservar: pueden ser valores estructurales, no faltantes.",
    "Conservar todos los flujos; un extremo no demuestra error ni ataque.",
    "No sobrescribir n_atipicos; guardar la revisión de las 12 variables por separado.")
)
guardar_tabla(decisiones, "decisiones_calidad")

# 4. Descriptiva de las 47 numéricas; se incluyen dpkts y las versiones log_.
numericas <- diccionario$nombre[diccionario$tipo == "numérica"]
resumen <- data.frame()
for (variable in numericas) {
  x <- base[[variable]]
  q <- quantile(x, c(0, .25, .5, .75, 1), na.rm = TRUE, names = FALSE, type = 7)
  resumen <- rbind(resumen, data.frame(variable = variable, media = mean(x, na.rm = TRUE),
    desvio = sd(x, na.rm = TRUE), minimo = q[1], q1 = q[2], mediana = q[3], q3 = q[4],
    maximo = q[5], ceros = sum(x == 0, na.rm = TRUE)))
}
guardar_tabla(resumen, "resumen_numerico")
print(resumen)

categoricas <- diccionario$nombre[diccionario$tipo == "categórica"]
frecuencias <- data.frame()
for (variable in categoricas) {
  tabla <- table(base[[variable]], useNA = "ifany")
  frecuencias <- rbind(frecuencias, data.frame(variable = variable, categoria = names(tabla),
    frecuencia = as.integer(tabla), porcentaje = 100 * as.integer(tabla) / nrow(base)))
```

```r
}
guardar_tabla(frecuencias, "frecuencias_categoricas")

# 5. Piloto con psych: Pearson, Bartlett, KMO y MSA por variable.
correlaciones <- cor(X_num, method = "pearson")
kmo <- KMO(correlaciones)
bartlett <- cortest.bartlett(correlaciones, n = nrow(X_num))
msa <- data.frame(variable = names(kmo$MSAi), msa = as.numeric(kmo$MSAi))
piloto <- data.frame(determinante = det(correlaciones), kmo_global = kmo$MSA,
  msa_minima = min(kmo$MSAi), bartlett_chi2 = bartlett$chisq, bartlett_gl = bartlett$df,
  bartlett_p_calculado = bartlett$p.value,
  bartlett_p_informe = ifelse(bartlett$p.value < .001, "< 0,001", as.character(bartlett$p.value)))
guardar_tabla(data.frame(variable = rownames(correlaciones), correlaciones), "correlaciones")
guardar_tabla(msa, "msa_por_variable")
guardar_tabla(piloto, "piloto_supuestos")
print(piloto)
print(msa)
if (kmo$MSA < .6 || min(kmo$MSAi) < .5 || det(correlaciones) <= 1e-5)
  warning("Revisar el piloto; no se cambia X_num automáticamente.")

# 6. Congelar X_cat: cuatro categorías existentes y tres discretizaciones.
# Los terciles no usan la etiqueta de ataque para elegir los cortes.
cortes_dur <- unname(quantile(base$dur, c(1/3, 2/3), type = 7))
cortes_bytes <- unname(quantile(base$sbytes, c(1/3, 2/3), type = 7))
X_cat <- as.data.frame(base[c("tipo_trafico", "proto_g", "service_g", "state_g")])
X_cat$respuesta_destino <- ifelse(base$dpkts == 0, "sin_respuesta", "con_respuesta")
X_cat$duracion_banda <- cut(base$dur, c(-Inf, cortes_dur, Inf), labels = c("corta", "media", "larga"))
X_cat$bytes_origen_banda <- cut(base$sbytes, c(-Inf, cortes_bytes, Inf), labels = c("bajos", "medios", "
    ↪ altos"))
X_cat[] <- lapply(X_cat, factor)
frecuencias_cat <- data.frame()
for (variable in names(X_cat)) {
  tabla <- table(X_cat[[variable]])
  frecuencias_cat <- rbind(frecuencias_cat, data.frame(variable = variable,
    categoria = names(tabla), frecuencia = as.integer(tabla), porcentaje = 100 * as.integer(tabla) / nrow(
    ↪ base)))
}
stopifnot(!anyNA(X_cat), all(frecuencias_cat$porcentaje >= 5))
guardar_tabla(frecuencias_cat, "frecuencias_X_cat")
guardar_tabla(data.frame(variable = c("dur", "sbytes"), corte_1 = c(cortes_dur[1], cortes_bytes[1]),
  corte_2 = c(cortes_dur[2], cortes_bytes[2])), "cortes_discretizacion")
guardar_tabla(data.frame(conjunto = c(rep("X_num", ncol(X_num)), rep("X_cat", ncol(X_cat))),
  variable = c(names(X_num), names(X_cat))), "conjuntos_congelados")

# 7. Guardar la base común y los conjuntos sin crear una carpeta datos ni archivos md.
stopifnot(identical(base, base_original))
saveRDS(base, "resultados/base_limpia.rds")
saveRDS(list(X_num = X_num, X_cat = X_cat, id = base$id, semilla = 42,
  cortes_dur = cortes_dur, cortes_bytes = cortes_bytes,
  estado_X_cat = "Congelado para los análisis de la Parte 2"), "resultados/conjuntos_congelados.rds")
cat("Base y conjuntos guardados en resultados. Excel original conservado.\n")
cat("Gráficos: ejecutar en Python el archivo python/graficos_etapa0.py.\n")
```

`R/01_acp.R` · Etapa 1: análisis de componentes principales

```r
# ETAPA 1. ACP sobre las 12 log_, desde la carpeta principal del proyecto.
set.seed(42)
base <- readRDS("resultados/base_limpia.rds")
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_num <- conjuntos$X_num
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")

# Estandarizar evita que las mayores varianzas determinen las componentes.
acp <- prcomp(X_num, scale. = TRUE)
acp_sin_escalar <- prcomp(X_num, scale. = FALSE)
autovalores <- acp$sdev^2
tabla_autovalores <- data.frame(componente = seq_along(autovalores), autovalor = autovalores,
  porcentaje = 100 * autovalores / sum(autovalores),
  acumulado = 100 * cumsum(autovalores) / sum(autovalores))
guardar(tabla_autovalores, "acp_autovalores")
```

```r
guardar(data.frame(variable = names(X_num), varianza = sapply(X_num, var)), "acp_varianzas")
guardar(data.frame(version = c("estandarizado", "sin_estandarizar"),
  porcentaje_CP1 = c(100 * autovalores[1] / sum(autovalores),
    100 * acp_sin_escalar$sdev[1]^2 / sum(acp_sin_escalar$sdev^2))), "acp_comparacion_escala")

# Kaiser sugiere tres; se contrasta el resultado con el codo y el significado de los ejes.
n_componentes <- sum(autovalores > 1)
correlaciones <- cor(X_num, acp$x)
contribuciones <- 100 * acp$rotation^2
cos2 <- correlaciones^2
guardar(data.frame(variable = rownames(correlaciones), correlaciones[, 1:n_componentes]), "acp_
    ↪ correlaciones")
guardar(data.frame(variable = rownames(contribuciones), contribuciones[, 1:n_componentes]), "acp_
    ↪ contribuciones")
guardar(data.frame(variable = rownames(cos2), cos2[, 1:n_componentes]), "acp_cos2")
puntajes <- acp$x[, 1:n_componentes, drop = FALSE]
guardar(data.frame(id = base$id, tipo_trafico = base$tipo_trafico, puntajes), "acp_puntajes")

# Distancia en el espacio retenido, ajustada por la varianza de cada componente.
distancia <- rowSums(sweep(puntajes, 2, acp$sdev[1:n_componentes], "/")^2)
extremos <- order(distancia, decreasing = TRUE)[1:3]
guardar(data.frame(base[extremos, c("id", "tipo_trafico", "dur", "spkts", "dpkts", "sbytes", "dbytes")],
  puntajes[extremos, , drop = FALSE], distancia = distancia[extremos]), "acp_extremos")
saveRDS(list(modelo = acp, n_componentes = n_componentes, puntajes = puntajes), "resultados/acp.rds")
print(tabla_autovalores)
cat("Componentes retenidas:", n_componentes, "\n")
```

R/02_af.R · Etapa 2: análisis factorial

```r
# ETAPA 2. Análisis factorial exploratorio y revisión de soluciones impropias (casos Heywood).
# La comparación completa de 3 vs 4 factores x métodos x rotaciones está en R/aux_diagnostico_af.R.
library(psych)
set.seed(42)
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_num <- conjuntos$X_num
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")
R <- cor(X_num)
kmo <- KMO(R)
bartlett <- cortest.bartlett(R, n = nrow(X_num))
stopifnot(det(R) > .00001, kmo$MSA >= .6, min(kmo$MSAi) >= .5, bartlett$p.value < .05)

# 1. Cuántos factores: evidencia gráfica (análisis paralelo) y analítica (Kaiser sobre la
# matriz reducida). El análisis paralelo no se dibuja en R: se exporta para Matplotlib.
options(mc.cores = 1)
paralelo <- fa.parallel(X_num, fa = "both", fm = "minres", n.iter = 100,
  quant = .95, plot = FALSE)
guardar(data.frame(dimension = seq_along(paralelo$fa.values),
  factor_observado = paralelo$fa.values, factor_simulado = paralelo$fa.sim,
  componente_observado = paralelo$pc.values, componente_simulado = paralelo$pc.sim), "af_paralelo")

# Kaiser aplicado a la matriz reducida (comunalidades iniciales SMC en la diagonal): es el
# criterio coherente con el AF, porque trabaja solo con la varianza común.
Rreducida <- R
diag(Rreducida) <- smc(R)
autovalores_reducida <- eigen(Rreducida, symmetric = TRUE, only.values = TRUE)$values
n_kaiser <- sum(autovalores_reducida > 1)

# Se retienen 3 factores aunque el análisis paralelo sugiere 4. Motivos (ver docs/validacion_decisiones.md):
#  - Con n = 5000 el análisis paralelo detecta cualquier estructura por encima del azar; el 4.º
#    autovalor observado (0,127) es 10 veces menor que 1 y el codo cae después del 3.º.
#  - Kaiser sobre la matriz reducida y el ACP (Etapa 1) dan 3, igual que la hipótesis H1.
#  - El 4.º factor solo parte el bloque ct_* en dos: queda con 2 variables fuertes (ct_srv_*),
#    lo que es síntoma de sobreextracción, y esos dos subfactores correlacionan 0,62 en oblimin.
n_paralelo <- paralelo$nfact
n_factores <- n_kaiser
stopifnot(n_factores == 3)

# 2. Caso Heywood. Se contrastan estimadores sobre los mismos 3 factores y la misma rotación.
# Un estimador "falla" si alguna comunalidad llega a 1 o más (unicidad <= 0.005, el límite de ML).
# Los avisos de psych ("ultra-Heywood") son esperables aquí: se informan, no se ocultan.
```

```r
ajustar <- function(metodo) suppressWarnings(fa(X_num, nfactors = n_factores, fm = metodo,
  rotate = "varimax", scores = "regression", n.obs = nrow(X_num)))
metodos <- c("minres", "pa", "ml", "gls")
fits <- lapply(setNames(metodos, metodos), ajustar)
diagnostico <- do.call(rbind, lapply(metodos, function(m) {
  f <- fits[[m]]
  data.frame(metodo = m, n_factores = n_factores, comunalidad_max = max(f$communality),
    unicidad_min = min(f$uniquenesses), variable_extrema = names(which.min(f$uniquenesses)),
    heywood = any(f$uniquenesses <= .005), rmsr = f$rms)
}))
guardar(diagnostico, "af_diagnostico_minres")
print(diagnostico)

# 3. Estimador final: GLS (mínimos cuadrados generalizados) con varimax.
# minres, pa y ML dan comunalidad ~1 (o mayor) para log_spkts; GLS la estima en torno a 0,87
# y todas las unicidades quedan lejos de cero. Es un remedio estándar ante un Heywood
# (cambiar de estimador) que no obliga a sacar una variable de X_num ni a fijar límites arbitrarios.
af <- fits[["gls"]]
cargas <- unclass(af$loadings)
# Se ordenan los factores por varianza explicada (psych ya los entrega así) y se renombran F1..F3.
colnames(cargas) <- paste0("F", seq_len(n_factores))
colnames(af$scores) <- paste0("Factor", seq_len(n_factores))

# Nombres: cada factor toma el bloque (H1) de las variables que más pesan en él.
bloque_h1 <- c(log_spkts = "Volumen", log_sbytes = "Volumen", log_dbytes = "Volumen",
  log_dur = "Temporización", log_sinpkt = "Temporización", log_sjit = "Temporización",
  log_synack = "Temporización", log_ct_srv_src = "Repetición de conexiones",
  log_ct_srv_dst = "Repetición de conexiones", log_ct_src_ltm = "Repetición de conexiones",
  log_ct_src_dport_ltm = "Repetición de conexiones", log_ct_dst_sport_ltm = "Repetición de conexiones")
asignado <- colnames(cargas)[apply(abs(cargas), 1, which.max)]
nombres_factores <- sapply(colnames(cargas), function(f) {
  b <- unique(bloque_h1[rownames(cargas)[asignado == f]])
  stopifnot(length(b) == 1) # si un factor mezcla bloques, hay que revisar el nombre a mano
  b
})

patron <- cargas
patron[abs(patron) < .4] <- NA # la consigna pide ocultar cargas menores a 0,40
diferencia <- apply(abs(cargas), 1, function(x) { x <- sort(x, decreasing = TRUE); x[1] - x[2] })
n_cargas <- rowSums(abs(cargas) >= .4)
comunalidad <- as.numeric(af$communality)
unicidad <- as.numeric(af$uniquenesses)
guardar(data.frame(variable = rownames(cargas), cargas, comunalidad = comunalidad,
  unicidad = unicidad, diferencia = diferencia, cargas_mayores_040 = n_cargas), "af_cargas_completas")
guardar(data.frame(variable = rownames(cargas), patron, comunalidad = comunalidad,
  unicidad = unicidad), "af_patron")
guardar(data.frame(factor = colnames(cargas), nombre = unname(nombres_factores)), "af_nombres_factores")
guardar(data.frame(factor = colnames(cargas), suma_cargas2 = colSums(cargas^2),
  porcentaje_total = 100 * colSums(cargas^2) / ncol(X_num),
  porcentaje_comun = 100 * colSums(cargas^2) / sum(cargas^2)), "af_varianza")
residuos <- R - tcrossprod(cargas)
rmsr <- sqrt(mean(residuos[lower.tri(residuos)]^2))
guardar(data.frame(metodo = "gls", n_factores = n_factores, n_sugerido_paralelo = n_paralelo,
  n_kaiser_reducida = n_kaiser, rmsr = rmsr, tli = af$TLI, rmsea = af$RMSEA[["RMSEA"]],
  chi2 = unname(af$STATISTIC), gl = af$dof, p_modelo = af$PVAL,
  comunalidad_min = min(comunalidad), unicidad_min = min(unicidad),
  unicidades_limite = sum(unicidad <= .00501),
  cruces_ambiguos = sum(n_cargas > 1 & diferencia < .2)), "af_ajuste")
guardar(data.frame(id = conjuntos$id, af$scores), "af_puntajes")
saveRDS(list(modelo = af, n_factores = n_factores, cargas = cargas, paralelo = paralelo,
  metodo = "gls", nombres = nombres_factores, diagnostico = diagnostico), "resultados/af.rds")
print(af$loadings, cutoff = .4)
print(af$uniquenesses)
```

`R/aux_diagnostico_af.R` · Etapa 2 (auxiliar): diagnóstico del caso Heywood

```r
# AUXILIAR (no forma parte de ejecutar_todo.R). Diagnóstico del caso Heywood en el AF.
# Ejecutar desde la raíz del repo:  Rscript R/aux_diagnostico_af.R
# Compara 3 vs 4 factores x (minres, ml, pa) x (varimax, oblimin) y guarda
# resultados/tablas/af_diagnostico_comparativo.csv y af_diagnostico_autovalores.csv.
# Sirve de evidencia para justificar cuántos factores retiene R/02_af.R.
```

```r
library(psych)
set.seed(42)
X <- readRDS("resultados/conjuntos_congelados.rds")$X_num
R <- cor(X)
p <- ncol(X)

# Bloques esperados por H1 (docs/etapa0_pregunta_hipotesis.md); solo para chequear interpretabilidad.
bloque_h1 <- c(log_spkts = "vol", log_sbytes = "vol", log_dbytes = "vol",
  log_dur = "tmp", log_sinpkt = "tmp", log_sjit = "tmp", log_synack = "tmp",
  log_ct_srv_src = "ctx", log_ct_srv_dst = "ctx", log_ct_src_ltm = "ctx",
  log_ct_src_dport_ltm = "ctx", log_ct_dst_sport_ltm = "ctx")

# 1. Autovalores: ¿es sustantivo el 4.º factor?
# Matriz reducida = R con la diagonal reemplazada por las comunalidades iniciales (SMC).
smc_ <- smc(R)
Rred <- R; diag(Rred) <- smc_
av_red <- eigen(Rred, symmetric = TRUE, only.values = TRUE)$values
av_cor <- eigen(R, symmetric = TRUE, only.values = TRUE)$values
set.seed(42)
pa <- suppressWarnings(fa.parallel(X, fa = "both", fm = "minres", n.iter = 100,
  quant = .95, plot = FALSE))
# Con n = 5000 el análisis paralelo detecta casi cualquier cosa por encima del azar:
# se agrega la varianza que aporta cada autovalor (autovalor reducido / suma de positivos).
aut <- data.frame(factor = 1:p, autovalor_R = av_cor, autovalor_reducida_smc = av_red,
  pct_var_total = 100 * av_cor / p,
  pct_var_comun = 100 * pmax(av_red, 0) / sum(pmax(av_red, 0)),
  fa_observado = pa$fa.values, fa_simulado = pa$fa.sim)
write.csv(aut, "resultados/tablas/af_diagnostico_autovalores.csv", row.names = FALSE, fileEncoding = "UTF-8
    ↪ ")
cat("\n== Autovalores (los 6 primeros) ==\n"); print(round(aut[1:6, ], 3), row.names = FALSE)
cat("Sugerencia del análisis paralelo (minres):", pa$nfact, "factores.\n")
cat("Kaiser sobre la matriz reducida (autovalor > 1):", sum(av_red > 1), "factores.\n")
cat("Autovalores reducidos > 0:", sum(av_red > 0), "\n")

# 2. Comparativo de soluciones
filas <- list()
for (k in 3:4) for (m in c("minres", "ml", "pa", "gls")) for (rot in c("varimax", "oblimin")) {
  aviso <- character()
  f <- withCallingHandlers(
    tryCatch(fa(X, nfactors = k, fm = m, rotate = rot, n.obs = nrow(X)), error = function(e) NULL),
    warning = function(w) { aviso <<- c(aviso, conditionMessage(w)); invokeRestart("muffleWarning") })
  if (is.null(f)) next
  L <- unclass(f$loadings)
  com <- f$communality
  uni <- f$uniquenesses
  ord <- t(apply(abs(L), 1, sort, decreasing = TRUE))
  dif <- ord[, 1] - ord[, 2]
  n04 <- rowSums(abs(L) >= .4)
  # Interpretabilidad: cada variable al factor de mayor carga; ¿cada factor agrupa un solo bloque H1?
  asig <- apply(abs(L), 1, which.max)
  # Cuántos bloques de H1 quedan recuperados exactamente (un factor = un bloque completo).
  grupos <- split(rownames(L), asig)
  puros <- sum(sapply(grupos, function(v) {
    b <- unique(bloque_h1[v]); length(b) == 1 && length(v) == sum(bloque_h1 == b) }))
  prop <- f$Vaccounted
  filas[[length(filas) + 1]] <- data.frame(
    n_factores = k, metodo = m, rotacion = rot,
    heywood = any(com >= 1 | uni <= .005),
    comunalidad_max = max(com), unicidad_min = min(uni),
    rmsr = f$rms, tli = f$TLI, rmsea = f$RMSEA[1],
    pct_var_acum = 100 * tail(prop["Cumulative Var", ], 1),
    sin_carga_040 = sum(n04 == 0),
    cruces_dif_menor_020 = sum(n04 > 1 & dif < .2),
    bloques_H1_recuperados = puros,
    avisos = length(aviso))
}
comp <- do.call(rbind, filas)
write.csv(comp, "resultados/tablas/af_diagnostico_comparativo.csv", row.names = FALSE, fileEncoding = "UTF
    ↪ -8")
cat("\n== Comparativo (varimax y oblimin) ==\n")
print(format(comp, digits = 3), row.names = FALSE)

# 3. Estructura de la solución de 3 y 4 factores con minres/varimax (la del análisis paralelo)
```

```r
for (k in 3:4) {
  f <- suppressWarnings(fa(X, nfactors = k, fm = "minres", rotate = "varimax"))
  cat("\n== minres + varimax,", k, "factores (cargas < 0,40 ocultas) ==\n")
  print(f$loadings, cutoff = .4, digits = 2)
  cat("Comunalidades:\n"); print(round(f$communality, 3))
}
# En 4 factores, ¿cuánto aporta el 4.º y qué hace? Correlación entre variables ct_* de cada puerto/servicio:
cat("\nCorrelaciones dentro del bloque ct_*:\n"); print(round(R[grep("ct_", rownames(R)), grep("ct_",
    ↪ colnames(R))], 2))
cat("\nCorrelación log_spkts con log_sbytes:", round(R["log_spkts", "log_sbytes"], 3),
    "| con log_dbytes:", round(R["log_spkts", "log_dbytes"], 3), "\n")

# 4. Factores correlacionados (oblimin, ML): ¿los dos subfactores ct_* del 4.º factor son separables?
f4o <- suppressWarnings(fa(X, nfactors = 4, fm = "ml", rotate = "oblimin"))
cat("\nCorrelación entre factores (ML, oblimin, 4 factores); los dos subfactores ct_* son los de mayor
    ↪ correlación:\n")
print(round(f4o$Phi, 2))

# 5. Sensibilidad (SOLO informativa; X_num no se modifica): ¿desaparece el Heywood sin log_spkts?
Xs <- X[, names(X) != "log_spkts"]
cat("\nSensibilidad sin log_spkts, 3 factores varimax:\n")
for (m in c("minres", "ml", "pa", "gls")) {
  f <- suppressWarnings(fa(Xs, nfactors = 3, fm = m, rotate = "varimax"))
  cat(sprintf("  %-7s comunalidad max = %.3f | unicidad min = %.3f | RMSR = %.3f\n", m,
              max(f$communality), min(f$uniquenesses), f$rms))
}
```

R/03_cluster.R · Etapa 3: conglomerados de observaciones y de variables

```r
# ETAPA 3. Ward y k-means sobre las 12 log_ estandarizadas.
library(cluster)
set.seed(42)
base <- readRDS("resultados/base_limpia.rds")
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_num <- conjuntos$X_num
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")
Z <- scale(X_num)
distancias <- dist(Z, method = "euclidean")
jerarquico <- hclust(distancias, method = "ward.D2")

# Codo: suma de cuadrados dentro de grupos. Silhouette sobre los 5.000 flujos.
metricas <- data.frame()
modelos_km <- list()
for (k in 2:8) {
  set.seed(42)
  km <- kmeans(Z, centers = k, nstart = 25, iter.max = 100)
  ward <- cutree(jerarquico, k)
  wss_ward <- 0
  for (g in 1:k) {
    grupo <- Z[ward == g, , drop = FALSE]
    wss_ward <- wss_ward + sum(sweep(grupo, 2, colMeans(grupo), "-")^2)
  }
  metricas <- rbind(metricas, data.frame(k = k, wss_ward = wss_ward, wss_kmeans = km$tot.withinss,
    silhouette_ward = mean(silhouette(ward, distancias)[, 3]),
    silhouette_kmeans = mean(silhouette(km$cluster, distancias)[, 3])))
  modelos_km[[as.character(k)]] <- km
  cat("Evaluado k =", k, "\n")
}
guardar(metricas, "cluster_metricas")

# Mayor silhouette de k-means entre los candidatos; el informe contrasta con el codo y Ward.
k_elegido <- metricas$k[which.max(metricas$silhouette_kmeans)]
km <- modelos_km[[as.character(k_elegido)]]
ward <- cutree(jerarquico, k_elegido)
guardar(as.data.frame.matrix(table(Ward = ward, Kmeans = km$cluster)), "cluster_cruce_ward_kmeans")

# Comparación de particiones sin depender del número arbitrario de las etiquetas.
ari <- function(a, b) {
  t <- table(a, b)
  observado <- sum(choose(t, 2))
```

```r
  esperado <- sum(choose(rowSums(t), 2)) * sum(choose(colSums(t), 2)) / choose(sum(t), 2)
  maximo <- (sum(choose(rowSums(t), 2)) + sum(choose(colSums(t), 2))) / 2
  (observado - esperado) / (maximo - esperado)
}

# Los puntajes de ACP conservan la ponderación por sus autovalores: no se reescalan.
acp <- readRDS("resultados/acp.rds")
set.seed(42)
km_acp <- kmeans(acp$puntajes, centers = k_elegido, nstart = 25, iter.max = 100)
ward_acp <- cutree(hclust(dist(acp$puntajes), method = "ward.D2"), k_elegido)
guardar(as.data.frame.matrix(table(Kmeans_log = km$cluster, Kmeans_ACP = km_acp$cluster)), "cluster_cruce_
    ↪ acp")
guardar(as.data.frame.matrix(table(Ward_log = ward, Ward_ACP = ward_acp)), "cluster_cruce_ward_acp")
guardar(data.frame(k = k_elegido, ARI_ward_kmeans = ari(ward, km$cluster),
  ARI_kmeans_acp = ari(km$cluster, km_acp$cluster), ARI_ward_acp = ari(ward, ward_acp)), "cluster_
    ↪ comparaciones")

originales <- sub("^log_", "", names(X_num))
guardar(aggregate(base[originales], list(cluster = km$cluster), mean), "cluster_medias_originales")
guardar(aggregate(as.data.frame(Z), list(cluster = km$cluster), mean), "cluster_medias_estandarizadas")
guardar(data.frame(cluster = 1:k_elegido, n = as.integer(table(km$cluster)),
  porcentaje = 100 * as.integer(table(km$cluster)) / nrow(base)), "cluster_tamanos")
guardar(as.data.frame(table(cluster = km$cluster, tipo_trafico = base$tipo_trafico)), "cluster_contexto")
guardar(aggregate(as.numeric(base$label == "ataque"), list(cluster = km$cluster), mean), "cluster_
    ↪ proporcion_ataques")

# Cluster de variables: distancia euclidiana entre sus columnas estandarizadas, Ward.D2.
jerarquico_vars <- hclust(dist(t(Z)), method = "ward.D2")
guardar(data.frame(variable = names(X_num), bloque = cutree(jerarquico_vars, 3)), "cluster_variables_
    ↪ bloques")
guardar(data.frame(izquierda = jerarquico$merge[, 1], derecha = jerarquico$merge[, 2],
  altura = jerarquico$height), "cluster_dendrograma")
guardar(data.frame(izquierda = jerarquico_vars$merge[, 1], derecha = jerarquico_vars$merge[, 2],
  altura = jerarquico_vars$height), "cluster_dendrograma_variables")
base$cluster <- factor(km$cluster)
base$cluster_ward <- factor(ward)
base$cluster_acp <- factor(km_acp$cluster)
saveRDS(base, "resultados/base_con_clusters.rds")
guardar(data.frame(id = base$id, cluster = km$cluster, cluster_ward = ward,
  cluster_acp = km_acp$cluster), "cluster_asignaciones")
saveRDS(list(k = k_elegido, kmeans = km, jerarquico = jerarquico,
  jerarquico_variables = jerarquico_vars), "resultados/clustering.rds")
print(metricas)
cat("k elegido:", k_elegido, "\n")
```

`R/04_acm.R` · Etapa 4: correspondencias simple y múltiples

```r
# ETAPA 4. ACS, ACM y proyección suplementaria del cluster.
library(ca)
library(FactoMineR)
set.seed(42)
base <- readRDS("resultados/base_con_clusters.rds")
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_cat <- conjuntos$X_cat
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")

# Las siete variables activas ya incluyen tres numéricas discretizadas en Etapa 0.
# Sus modalidades superan 5%; no se fusionan categorías adicionales.
frecuencias <- read.csv("resultados/tablas/frecuencias_X_cat.csv")
stopifnot(all(frecuencias$porcentaje >= 5), all(sapply(X_cat, is.factor)))

# ACS: tipo de tráfico y estado agrupado, para estudiar la tercera hipótesis.
tabla <- table(X_cat$tipo_trafico, X_cat$state_g)
acs <- ca::ca(tabla, nd = 2)
chi <- chisq.test(tabla, correct = FALSE)
guardar(data.frame(tipo_trafico = rownames(tabla), unclass(tabla)), "acs_contingencia")
guardar(data.frame(tipo_trafico = rownames(tabla), unclass(100 * prop.table(tabla, 1))), "acs_porcentajes_
    ↪ fila")
guardar(data.frame(dimension = seq_along(acs$sv), autovalor = acs$sv^2,
  porcentaje = 100 * acs$sv^2 / sum(acs$sv^2)), "acs_inercia")
```

```r
guardar(data.frame(chi2 = unname(chi$statistic), gl = unname(chi$parameter), p = chi$p.value,
  cramer_v = sqrt(unname(chi$statistic) / (sum(tabla) * min(nrow(tabla) - 1, ncol(tabla) - 1))),
  frecuencia_esperada_min = min(chi$expected)), "acs_asociacion")
guardar(data.frame(categoria = rownames(tabla),
  sweep(acs$rowcoord[, 1:2, drop = FALSE], 2, acs$sv[1:2], "*")), "acs_filas")
guardar(data.frame(categoria = colnames(tabla),
  sweep(acs$colcoord[, 1:2, drop = FALSE], 2, acs$sv[1:2], "*")), "acs_columnas")

# Cluster es suplementario: no interviene en la construcción de los ejes.
datos_acm <- data.frame(X_cat, cluster = base$cluster)
Q <- ncol(X_cat)
J <- sum(sapply(X_cat, nlevels))
acm <- FactoMineR::MCA(datos_acm, quali.sup = ncol(datos_acm), ncp = J - Q, graph = FALSE)
ajustado <- ca::mjca(X_cat, nd = NA, lambda = "adjusted")
indicador <- ca::mjca(X_cat, nd = NA, lambda = "indicator")
# ca 0.72 entrega valores singulares: al cuadrarlos se obtienen los autovalores.
stopifnot(max(abs(indicador$sv^2 - acm$eig[, 1])) < 1e-8)
autovalores <- data.frame(dimension = seq_len(nrow(acm$eig)), autovalor = acm$eig[, 1],
  porcentaje_crudo = acm$eig[, 2], acumulado_crudo = acm$eig[, 3],
  autovalor_ajustado = 0, porcentaje_ajustado = 0)
n_aj <- length(ajustado$sv)
autovalores$autovalor_ajustado[1:n_aj] <- ajustado$sv^2
autovalores$porcentaje_ajustado[1:n_aj] <- 100 * ajustado$inertia.e
autovalores$acumulado_ajustado <- cumsum(autovalores$porcentaje_ajustado)
guardar(autovalores, "acm_autovalores")
guardar(data.frame(Q = Q, J = J, inercia_cruda = J / Q - 1,
  umbral = 1 / Q, ejes_sobre_umbral = sum(acm$eig[, 1] > 1 / Q),
  inercia_total_ajustada = ajustado$inertia.t,
  porcentaje_plano_ajustado = sum(autovalores$porcentaje_ajustado[1:2])), "acm_resumen")

# Se conservan todos los resultados; el informe interpreta el plano de dos dimensiones.
categorias <- ajustado$levelnames
coord <- data.frame(categoria = categorias, acm$var$coord)
contrib <- data.frame(categoria = categorias, acm$var$contrib)
cos2 <- data.frame(categoria = categorias, acm$var$cos2)
guardar(coord, "acm_categorias_coordenadas")
guardar(contrib, "acm_categorias_contribuciones")
guardar(cos2, "acm_categorias_cos2")
guardar(data.frame(variable = rownames(acm$var$eta2), acm$var$eta2), "acm_eta2")
guardar(data.frame(cluster = levels(base$cluster), acm$quali.sup$coord), "acm_cluster_suplementario")
guardar(data.frame(cluster = levels(base$cluster), acm$quali.sup$cos2), "acm_cluster_cos2")
guardar(data.frame(id = base$id, cluster = base$cluster, acm$ind$coord[, 1:2]), "acm_individuos")

# Cruces para interpretar tres asociaciones con frecuencias observadas, sin causalidad.
for (variable in c("respuesta_destino", "duracion_banda", "bytes_origen_banda")) {
  t <- table(X_cat$tipo_trafico, X_cat[[variable]])
  guardar(data.frame(tipo_trafico = rownames(t), unclass(100 * prop.table(t, 1))), paste0("acm_cruce_",
    ↪ variable))
}
saveRDS(list(acs = acs, acm = acm, ajustado = ajustado, dimensiones_interpretadas = 2), "resultados/
    ↪ correspondencias.rds")
print(autovalores)
print(acm$var$eta2[, 1:2])
```

`R/05_integracion.R` · Etapa 6: integración de técnicas

```r
# ETAPA 6. Síntesis de resultados de las técnicas multivariadas.
set.seed(42)
base <- readRDS("resultados/base_con_clusters.rds")
X <- readRDS("resultados/conjuntos_congelados.rds")$X_num
acp <- readRDS("resultados/acp.rds")
af <- readRDS("resultados/af.rds")
km <- readRDS("resultados/clustering.rds")
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")

# Evidencia de la hipótesis de asociaciones, con tamaños del efecto.
asociaciones <- data.frame()
for (variable in c("proto_g", "service_g", "state_g")) {
  t <- table(base$tipo_trafico, base[[variable]])
  prueba <- chisq.test(t, correct = FALSE)
```

```r
  asociaciones <- rbind(asociaciones, data.frame(variable = variable,
    chi2 = unname(prueba$statistic), gl = unname(prueba$parameter), p = prueba$p.value,
    cramer_v = sqrt(unname(prueba$statistic) / (sum(t) * min(dim(t) - 1)))))
}
guardar(asociaciones, "integracion_asociaciones")

# Todas las cifras de la tabla salen de los objetos y tablas ya calculados (nada escrito a mano).
# Formato argentino: coma decimal.
fmt <- function(x, d = 2) formatC(x, format = "f", digits = d, decimal.mark = ",")
leer <- function(nombre) read.csv(paste0("resultados/tablas/", nombre, ".csv"), fileEncoding = "UTF-8")
ari <- function(a, b) { # índice de Rand ajustado entre dos particiones
  t <- table(a, b)
  observado <- sum(choose(t, 2))
  esperado <- sum(choose(rowSums(t), 2)) * sum(choose(colSums(t), 2)) / choose(sum(t), 2)
  maximo <- (sum(choose(rowSums(t), 2)) + sum(choose(colSums(t), 2))) / 2
  (observado - esperado) / (maximo - esperado)
}

var_acp <- sum(acp$modelo$sdev[seq_len(acp$n_componentes)]^2) / sum(acp$modelo$sdev^2) * 100
ajuste_af <- leer("af_ajuste")
var_af <- sum(leer("af_varianza")$porcentaje_total)
heywood_final <- ajuste_af$unicidades_limite > 0 || ajuste_af$unicidad_min <= 0
v_cramer <- asociaciones$cramer_v[asociaciones$variable == "state_g"]
inercia_acm <- leer("acm_resumen")$porcentaje_plano_ajustado
cmp <- leer("cluster_comparaciones")
sil <- leer("cluster_metricas")
sil_k <- sil$silhouette_kmeans[sil$k == km$k]
# ¿Los bloques del cluster de variables coinciden con los factores del AF?
bloques_vars <- leer("cluster_variables_bloques")
factor_dominante <- apply(abs(af$cargas), 1, which.max)
ari_bloques_af <- ari(bloques_vars$bloque, factor_dominante[bloques_vars$variable])

integracion <- data.frame(
  tecnica = c("ACP", "AF", "Clustering", "ACS", "ACM"),
  estructura = c(
    paste0(acp$n_componentes, " componentes; ", fmt(var_acp), "% de varianza"),
    paste0(af$n_factores, " factores (", toupper(af$metodo), ", varimax); ", fmt(var_af),
      "% de la varianza total; ", ifelse(heywood_final, "con unicidad en el límite",
      paste0("sin caso Heywood (unicidad mínima ", fmt(ajuste_af$unicidad_min, 3), ")"))),
    paste0("Tres bloques de variables y ", km$k, " grupos de observaciones"),
    paste0("Tipo de tráfico por estado; V de Cramér = ", fmt(v_cramer, 3)),
    paste0("Plano de dos dimensiones; ", fmt(inercia_acm), "% de inercia ajustada")),
  interpretacion = c(
    "Ejes mixtos: magnitud del flujo frente a repetición (CP1), volumen de tráfico (CP2) y ritmo lento (CP3)",
    paste0(paste(af$nombres, collapse = ", "), " (cargas >= 0,40; cruces ambiguos = ",
      ajuste_af$cruces_ambiguos, ")"),
    paste0("El AF reconoce los bloques (ARI bloques de variables vs. factores = ", fmt(ari_bloques_af, 3),
      "); ARI Ward/KM = ", fmt(cmp$ARI_ward_kmeans, 3), ", silhouette = ", fmt(sil_k, 3),
      "; con puntajes de ACP el ARI frente a k-means es ", fmt(cmp$ARI_kmeans_acp, 3), " (las CP cambian fronteras)"),
    "Normal-CON_otros y Exploits-FIN coinciden con asociaciones del ACM",
    "Dim 1: conexión efímera sin respuesta frente a sesión TCP completa; Dim 2: DNS/UDP con estado CON frente a protocolo otros no establecido")
)
guardar(integracion, "integracion_tecnicas")
print(integracion)
```

R/06_logistica.R · Regresión logística

```r
# ETAPA 6. Regresión logística: ¿es ataque (1) o normal (0)?
# Ejecutar desde la carpeta principal del proyecto: Rscript R/06_logistica.R
#
# Diseño (decidido por el grupo/orquestador):
#  - Validación: partición estratificada por label, 70 % entrenamiento / 30 % prueba,
#    dentro de la muestra de 5.000 flujos. El testing set original de UNSW-NB15 no está
#    en el repositorio y habría que prepararlo igual que la muestra.
#  - Predictores: los 3 puntajes del ACP + proto_g, service_g, state_g.
#  - Prohibidos: TTL (sttl, dttl, ttl_origen, perfil_estado_ttl, ct_state_ttl: separación
#    perfecta por artefacto del laboratorio), tipo_trafico, attack_cat y n_atipicos
#    (son la respuesta "disfrazada" o derivan de ella).
```

```r
#  - El ACP se reajusta SOLO con el entrenamiento para que el conjunto de prueba no
#    influya en los puntajes (evita fuga de información).
library(MASS)   # confint() por verosimilitud perfilada para glm
library(pROC)
set.seed(42)

base <- readRDS("resultados/base_limpia.rds")
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_num <- conjuntos$X_num
acp_completo <- readRDS("resultados/acp.rds")
n_comp <- acp_completo$n_componentes   # 3 componentes (Kaiser), como en el informe
dir.create("resultados/tablas", recursive = TRUE, showWarnings = FALSE)
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/log_reg_", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")
coma <- function(x, d = 3) formatC(x, format = "f", digits = d)

# 1. Respuesta: ataque = 1, normal = 0 (en la base label es texto "ataque"/"normal").
stopifnot(all(base$label %in% c("ataque", "normal")))
base$y <- as.integer(base$label == "ataque")

# 2. Partición 70/30 estratificada por attack_cat (10 categorías), no solo por label.
# Por qué: la muestra de 5.000 se armó estratificando por attack_cat; volver a estratificar
# así respeta el diseño muestral y deja cada tipo de ataque (incluso los raros, como Worms
# con 6 flujos) en la misma proporción en entrenamiento y prueba. label queda balanceado
# automáticamente porque es función de attack_cat (Normal = normal, el resto = ataque).
# Dentro de cada categoría van round(0,7 * n) flujos al entrenamiento y el resto a prueba.
idx_train <- unlist(lapply(split(seq_len(nrow(base)), base$attack_cat), function(i)
  i[sample.int(length(i), size = round(0.7 * length(i)))]))
idx_train <- sort(idx_train)
idx_test <- setdiff(seq_len(nrow(base)), idx_train)
stopifnot(length(intersect(idx_train, idx_test)) == 0)
# Ninguna categoría puede quedar vacía en prueba ni en entrenamiento.
n_cat <- table(base$attack_cat, ifelse(seq_len(nrow(base)) %in% idx_train, "entrenamiento", "prueba"))
stopifnot(all(n_cat > 0), n_cat["Worms", "entrenamiento"] == 4, n_cat["Worms", "prueba"] == 2)
conjunto <- ifelse(seq_len(nrow(base)) %in% idx_train, "entrenamiento", "prueba")

# Resumen general (n y % de ataques por conjunto)
particion <- data.frame(conjunto = c("entrenamiento", "prueba"),
  n = c(length(idx_train), length(idx_test)),
  n_ataques = c(sum(base$y[idx_train]), sum(base$y[idx_test])),
  pct_ataques = c(100 * mean(base$y[idx_train]), 100 * mean(base$y[idx_test])))
guardar(particion, "particion")
# Comparación de proporciones por attack_cat entre ambos conjuntos
cats <- rownames(n_cat)[order(-rowSums(n_cat))]
comp_cat <- data.frame(attack_cat = cats, n_total = as.integer(rowSums(n_cat)[cats]),
  n_entrenamiento = as.integer(n_cat[cats, "entrenamiento"]), n_prueba = as.integer(n_cat[cats, "prueba"]),
  pct_entrenamiento = 100 * n_cat[cats, "entrenamiento"] / sum(n_cat[, "entrenamiento"]),
  pct_prueba = 100 * n_cat[cats, "prueba"] / sum(n_cat[, "prueba"]))
comp_cat$pct_dentro_entrenamiento <- 100 * comp_cat$n_entrenamiento / comp_cat$n_total
guardar(comp_cat, "particion_attack_cat")
# Ids de cada partición (id, attack_cat, label, conjunto)
guardar(data.frame(id = base$id, attack_cat = base$attack_cat, label = base$label,
  conjunto = conjunto), "particion_ids")
# Excel con las 68 columnas de la muestra (la hoja original NO se modifica):
# hojas entrenamiento, prueba y resumen. base es exactamente la hoja `muestra`.
if (requireNamespace("writexl", quietly = TRUE)) {
  writexl::write_xlsx(list(entrenamiento = as.data.frame(base[idx_train, setdiff(names(base), "y")]),
    prueba = as.data.frame(base[idx_test, setdiff(names(base), "y")]), resumen = comp_cat),
    "resultados/muestra_particion_70_30.xlsx")
} else cat("Paquete writexl no instalado: install.packages(\"writexl\") para exportar la partición\n")
print(particion); print(comp_cat)

# 3. ACP reajustado SOLO con el entrenamiento; la prueba se proyecta con predict(), que
# usa las medias y desvíos del entrenamiento.
acp_tr <- prcomp(X_num[idx_train, ], scale. = TRUE)
# Los signos de una componente son arbitrarios. Se alinean con el ACP del informe (5.000):
# si la carga de entrenamiento apunta al lado contrario, se invierte la componente.
signos <- sapply(1:n_comp, function(k)
  sign(sum(acp_tr$rotation[, k] * acp_completo$modelo$rotation[, k])))
cat("Signo relativo al ACP completo (1 = igual, -1 = invertido):", signos, "\n")
acp_tr$rotation[, 1:n_comp] <- sweep(acp_tr$rotation[, 1:n_comp, drop = FALSE], 2, signos, "*")
# Se recalculan los puntajes de entrenamiento con la rotación ya corregida.
nombres_cp <- paste0("PC", 1:n_comp)
```

```r
P_tr <- predict(acp_tr, X_num[idx_train, ])[, 1:n_comp]
P_te <- predict(acp_tr, X_num[idx_test, ])[, 1:n_comp]
colnames(P_tr) <- colnames(P_te) <- nombres_cp

# Comparación de cargas: entrenamiento vs. ACP con los 5.000.
dif_cargas <- acp_tr$rotation[, 1:n_comp] - acp_completo$modelo$rotation[, 1:n_comp]
comp_cargas <- data.frame(variable = rownames(acp_tr$rotation),
  setNames(as.data.frame(acp_tr$rotation[, 1:n_comp]), paste0(nombres_cp, "_entrenamiento")),
  setNames(as.data.frame(acp_completo$modelo$rotation[, 1:n_comp]), paste0(nombres_cp, "_completo")))
guardar(comp_cargas, "acp_cargas_comparacion")
resumen_cargas <- data.frame(componente = nombres_cp, signo_invertido = signos == -1,
  max_dif_absoluta = apply(abs(dif_cargas), 2, max),
  correlacion = sapply(1:n_comp, function(k)
    cor(acp_tr$rotation[, k], acp_completo$modelo$rotation[, k])),
  pct_varianza_entrenamiento = 100 * acp_tr$sdev[1:n_comp]^2 / sum(acp_tr$sdev^2))
guardar(resumen_cargas, "acp_cargas_resumen")
print(resumen_cargas)

# 4. Categóricas con categoría de referencia explícita: la más frecuente de cada una,
# así las demás se comparan contra la situación "típica" y el error estándar de la
# referencia es el menor posible: tcp (70,8 %), ninguno (60,0 %), FIN (69,3 %).
ref <- function(x, nivel) relevel(factor(x), ref = nivel)
datos <- data.frame(y = base$y, id = base$id, P = rbind(P_tr, P_te)[order(c(idx_train, idx_test)), ],
  proto_g = ref(base$proto_g, "tcp"), service_g = ref(base$service_g, "ninguno"),
  state_g = ref(base$state_g, "FIN"))
names(datos) <- sub("^P\\.", "", names(datos))
train <- datos[idx_train, ]
test <- datos[idx_test, ]
stopifnot(identical(train$id, base$id[idx_train]))

# 5. Dos modelos anidados. M1: solo los puntajes. M2: M1 + categóricas.
M1 <- glm(y ~ PC1 + PC2 + PC3, data = train, family = binomial)
aviso <- NULL
M2 <- withCallingHandlers(
  glm(y ~ PC1 + PC2 + PC3 + proto_g + service_g + state_g, data = train, family = binomial),
  warning = function(w) { aviso <<- c(aviso, conditionMessage(w)); invokeRestart("muffleWarning") })
if (!is.null(aviso)) cat("AVISO al ajustar M2:", aviso, "\n")

# 6. Comparación: AIC, devianza, test de razón de verosimilitud y AUC en prueba.
auc_prueba <- function(m) auc(roc(test$y, as.numeric(predict(m, test, type = "response")),
  levels = c(0, 1), direction = "<", quiet = TRUE))
mcfadden <- function(m) 1 - m$deviance / m$null.deviance
lr <- anova(M1, M2, test = "Chisq")
comparacion <- data.frame(modelo = c("M1: puntajes ACP", "M2: puntajes ACP + proto + service + state"),
  gl_residuales = c(M1$df.residual, M2$df.residual), parametros = c(length(coef(M1)), length(coef(M2))),
  AIC = c(AIC(M1), AIC(M2)), BIC = c(BIC(M1), BIC(M2)),
  devianza = c(deviance(M1), deviance(M2)),
  LR_chi2 = c(NA, lr$Deviance[2]), LR_gl = c(NA, lr$Df[2]), LR_p = c(NA, lr$`Pr(>Chi)`[2]),
  AUC_prueba = c(as.numeric(auc_prueba(M1)), as.numeric(auc_prueba(M2))),
  pseudoR2_McFadden = c(mcfadden(M1), mcfadden(M2)))
guardar(comparacion, "comparacion_modelos")
print(comparacion)

# 7. Elección: M2 si mejora el AIC y el LR es significativo (la ganancia no es azar);
# si no, M1 por parsimonia. La decisión sale de los números, no se fija a mano.
elegido <- if (AIC(M2) < AIC(M1) && lr$`Pr(>Chi)`[2] < 0.05) "M2" else "M1"
modelo <- if (elegido == "M2") M2 else M1
cat("Modelo elegido:", elegido, "\n")

# 8. Coeficientes, OR e IC 95 % por verosimilitud perfilada (más fiable que Wald cuando
# hay categorías raras o coeficientes grandes). Se incluye también el IC de Wald.
sm <- summary(modelo)$coefficients
ic_perfil <- suppressMessages(confint(modelo))
se <- sm[, 2]
coefs <- data.frame(termino = rownames(sm), estimacion = sm[, 1], error_estandar = se,
  z = sm[, 3], p = sm[, 4], OR = exp(sm[, 1]),
  OR_IC95_inf = exp(ic_perfil[, 1]), OR_IC95_sup = exp(ic_perfil[, 2]),
  OR_Wald_IC95_inf = exp(sm[, 1] - 1.96 * se), OR_Wald_IC95_sup = exp(sm[, 1] + 1.96 * se),
  tipo_IC = "verosimilitud perfilada (Wald en columnas aparte)")
guardar(coefs, "coeficientes")
# Los OR de PC1-PC3 son "por cada desvío (1 unidad estandarizada) del puntaje", porque
# el puntaje de cada componente tiene desvío = sdev de la componente: el OR por 1 desvío
# del puntaje es exp(beta * sdev). Se guarda también esa versión, que es la comparable.
```

```r
sdev_tr <- acp_tr$sdev[1:n_comp]
or_desvio <- data.frame(termino = nombres_cp, desvio_puntaje = sdev_tr,
  OR_por_unidad = exp(coef(modelo)[nombres_cp]),
  OR_por_desvio = exp(coef(modelo)[nombres_cp] * sdev_tr),
  OR_desvio_IC95_inf = exp(ic_perfil[nombres_cp, 1] * sdev_tr),
  OR_desvio_IC95_sup = exp(ic_perfil[nombres_cp, 2] * sdev_tr))
guardar(or_desvio, "or_por_desvio")
print(coefs[, 1:8]); print(or_desvio)

# 9. Desempeño en prueba con umbral 0,5. AUC con IC 95 % de DeLong.
p_te <- as.numeric(predict(modelo, test, type = "response"))
roc_te <- roc(test$y, p_te, levels = c(0, 1), direction = "<", quiet = TRUE)
ic_auc <- ci.auc(roc_te, method = "delong")
guardar(data.frame(AUC = as.numeric(auc(roc_te)), IC95_inf = ic_auc[1], IC95_sup = ic_auc[3],
  metodo_IC = "DeLong", modelo = elegido), "auc")
pred <- as.integer(p_te >= 0.5)
VP <- sum(pred == 1 & test$y == 1); VN <- sum(pred == 0 & test$y == 0)
FP <- sum(pred == 1 & test$y == 0); FN <- sum(pred == 0 & test$y == 1)
guardar(data.frame(real = c("normal", "normal", "ataque", "ataque"),
  predicho = c("normal", "ataque", "normal", "ataque"), n = c(VN, FP, FN, VP)), "confusion")
metricas <- data.frame(umbral = 0.5, exactitud = (VP + VN) / length(pred),
  sensibilidad = VP / (VP + FN), especificidad = VN / (VN + FP),
  precision = VP / (VP + FP), F1 = 2 * VP / (2 * VP + FP + FN))
guardar(metricas, "metricas")
print(metricas); print(ic_auc)

# Curva ROC (puntos para graficar) y probabilidades predichas en prueba.
guardar(data.frame(umbral = roc_te$thresholds, fpr = 1 - roc_te$specificities,
  tpr = roc_te$sensitivities), "roc_curva")
guardar(data.frame(id = test$id, y = test$y, p = p_te), "predicciones_prueba")

# 10. Calibración por deciles de probabilidad predicha (en prueba).
dec <- cut(p_te, breaks = unique(quantile(p_te, seq(0, 1, .1))), include.lowest = TRUE, labels = FALSE)
calib <- do.call(rbind, lapply(sort(unique(dec)), function(d) data.frame(decil = d,
  n = sum(dec == d), p_media = mean(p_te[dec == d]), prop_observada = mean(test$y[dec == d]))))
guardar(calib, "calibracion")
print(calib)

# 11. Multicolinealidad: VIF de cada columna de la matriz de diseño y correlaciones.
# (Las dummies de un mismo factor se correlacionan entre sí por construcción.)
mm <- model.matrix(modelo)[, -1, drop = FALSE]
vif <- sapply(seq_len(ncol(mm)), function(j) 1 / (1 - summary(lm(mm[, j] ~ mm[, -j]))$r.squared))
guardar(data.frame(termino = colnames(mm), VIF = vif), "vif")
guardar(data.frame(termino = colnames(mm), round(cor(mm), 4)), "correlaciones_predictores")
print(data.frame(termino = colnames(mm), VIF = round(vif, 2)))

# 12. Separación: coeficientes enormes, EE gigantes o probabilidades ajustadas en 0/1.
ajustadas <- fitted(modelo)
separacion <- data.frame(
  max_abs_coef = max(abs(coef(modelo))), max_error_estandar = max(se),
  ajustadas_menor_1e8 = sum(ajustadas < 1e-8), ajustadas_mayor_1_menos_1e8 = sum(ajustadas > 1 - 1e-8),
  aviso_glm = ifelse(is.null(aviso), "sin avisos", paste(unique(aviso), collapse = " | ")),
  convergio = modelo$converged, iteraciones = modelo$iter)
guardar(separacion, "separacion")
print(separacion)
# Tablas cruzadas categoría x respuesta en entrenamiento: celdas vacías delatarían separación.
cruces <- do.call(rbind, lapply(c("proto_g", "service_g", "state_g"), function(v) {
  t <- table(train[[v]], train$y)
  data.frame(variable = v, categoria = rownames(t), n_normal = t[, "0"], n_ataque = t[, "1"],
    pct_ataque = 100 * t[, "1"] / rowSums(t))
}))
guardar(cruces, "cruces_categoricas")
print(cruces)

# 13. Observaciones influyentes: distancia de Cook > 4/n.
cook <- cooks.distance(modelo)
umbral_cook <- 4 / nrow(train)
influyentes <- data.frame(umbral = umbral_cook, n_influyentes = sum(cook > umbral_cook),
  pct = 100 * mean(cook > umbral_cook), cook_max = max(cook), id_cook_max = train$id[which.max(cook)],
  n_cook_mayor_1 = sum(cook > 1))
guardar(influyentes, "influyentes")
print(influyentes)
```

```r
# 14. Guardar el modelo y lo necesario para reproducir las predicciones.
# Interpretación: la probabilidad predicha es relativa a ESTA base (≈ 52 % de ataques,
# sin duplicados, ataques sintéticos del laboratorio): se interpretan los odds ratios,
# no prevalencias absolutas. El OR de PCk es por cada desvío del puntaje.
saveRDS(list(modelo = modelo, elegido = elegido, M1 = M1, M2 = M2, acp_entrenamiento = acp_tr,
  idx_train = idx_train, idx_test = idx_test, ids_train = base$id[idx_train],
  ids_test = base$id[idx_test], semilla = 42), "resultados/logistica.rds")
cat("Listo. Modelo", elegido, "| AUC prueba:", coma(as.numeric(auc(roc_te))),
  "IC95 [", coma(ic_auc[1]), ",", coma(ic_auc[3]), "]\n")
```

`R/ejecutar_todo.R` · Ejecución completa del proyecto

```r
# Ejecutar desde la carpeta principal del repositorio.
# Paquetes (una sola vez):
# install.packages(c("readxl", "psych", "GPArotation", "cluster", "ca", "FactoMineR",
#                    "MASS", "pROC", "writexl"))
# Las figuras se crean luego con Python; ver Como_ejecutar.txt.
for (archivo in c("00_datos.R", "01_acp.R", "02_af.R", "03_cluster.R", "04_acm.R",
                  "05_integracion.R", "06_logistica.R")) {
  cat("\nEJECUTANDO", archivo, "\n")
  source(paste0("R/", archivo), encoding = "UTF-8")
}
```

Scripts de Python

`python/graficos_etapa0.py` · Gráficos de la Etapa 0

```python
"""Gráficos pastel de la Etapa 0. Ejecutar después de R/00_datos.R."""
from pathlib import Path

import matplotlib
matplotlib.use("Agg")  # Exportación reproducible: no abre ventanas ni requiere pantalla.
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.ticker import MaxNLocator, FuncFormatter, ScalarFormatter
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
TABLES = ROOT / "resultados" / "tablas"
FIGURES = ROOT / "resultados" / "figuras"
FIGURES.mkdir(parents=True, exist_ok=True)

def read(name):
    return pd.read_csv(TABLES / f"{name}.csv")

base = pd.read_excel(ROOT / "muestra_estratificada-simulado-de-ataques.xlsx", sheet_name="muestra")
diccionario = read("diccionario_variables")
conjuntos = read("conjuntos_congelados")
variables = conjuntos.loc[conjuntos.conjunto.eq("X_num"), "variable"].tolist()
crudas = [v.removeprefix("log_") for v in variables]
crudas.insert(2, "dpkts")  # Se describe la respuesta del destino, aunque no entra en X_num.
limites = read("atipicos_limites").set_index("variable")
frecuencias = read("frecuencias_X_cat")
correlaciones = read("correlaciones").set_index("variable")
msa = read("msa_por_variable")
piloto = read("piloto_supuestos").iloc[0]

plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
    "axes.titlesize": 11, "axes.labelsize": 10, "figure.facecolor": "white",
    "axes.facecolor": "white", "axes.spines.top": False, "axes.spines.right": False,
    "text.color": "#283C49", "axes.labelcolor": "#283C49",
    "xtick.color": "#465761", "ytick.color": "#465761"})
PASTEL = ["#C7DCEB", "#D7CCE8", "#CDE2D3", "#F0D4BE", "#EBCDD5"]
EDGE = "#556874"

def coma_decimal(fig):
    """Usar coma decimal en los ejes numéricos, como en el resto del informe."""
    for ax in fig.axes:
```

```
        for eje in (ax.xaxis, ax.yaxis):
            if isinstance(eje.get_major_formatter(), ScalarFormatter):
                eje.set_major_formatter(FuncFormatter(lambda v, _: f"{v:g}".replace(".", ",")))

def guardar(fig, name):
    coma_decimal(fig)
    fig.savefig(FIGURES / f"{name}.png", dpi=200, bbox_inches="tight", facecolor="white")
    plt.close(fig)

def ejes_panel(cantidad=12):
    filas = (cantidad + 3) // 4
    return plt.subplots(filas, 4, figsize=(15, filas * 3.7), layout="constrained")

def nota(fig, texto):
    fig.supxlabel(texto, fontsize=9, color="#465761")

# Se usan las bisagras calculadas por R; bxp evita diferencias de cuartiles por empates.
fig, axes = ejes_panel()
for i, (ax, v) in enumerate(zip(axes.flat, variables)):
    row = limites.loc[v]
    x = base[v].to_numpy()
    extremos = x[(x < row.limite_inferior) | (x > row.limite_superior)]
    stats = {"med": row.mediana, "q1": row.q1_tukey, "q3": row.q3_tukey,
        "whislo": row.bigote_inferior, "whishi": row.bigote_superior, "fliers": extremos}
    ax.bxp([stats], positions=[1], widths=.45, patch_artist=True, manage_ticks=False,
        boxprops={"facecolor": PASTEL[i // 4], "edgecolor": EDGE},
        medianprops={"color": "#283C49", "linewidth": 1.5},
        whiskerprops={"color": EDGE}, capprops={"color": EDGE},
        flierprops={"marker": ".", "markersize": 3, "alpha": .28,
                    "markeredgecolor": EDGE, "markerfacecolor": EDGE})
    ax.set_title(f"{v}\n{int(row.n_atipicos):,} atípicos".replace(",", "."))
    ax.set_xticks([])
    ax.set_ylabel("log(x + 1)")
    ax.grid(axis="y", color="#E5E9EC", linewidth=.5)
fig.suptitle("Control de atípicos · 12 variables seleccionadas", fontsize=17)
nota(fig, "5.000 flujos · bisagras de Tukey ± 1,5 RIC · escalas independientes · se conservan todos los
    ↪ registros")
guardar(fig, "01_boxplots")

# Histogramas: todos los datos, sin recortar colas; bins FD con tope visual de 80.
def histogramas(lista, nombre, titulo, escala):
    fig, axes = ejes_panel(len(lista))
    for i, (ax, v) in enumerate(zip(axes.flat, lista)):
        x = base[v].to_numpy()
        q1, q3 = np.quantile(x, [.25, .75])
        ancho = 2 * (q3 - q1) / np.cbrt(len(x))
        n_bins = int(np.clip(np.ceil(np.ptp(x) / ancho), 10, 80)) if ancho > 0 else 20
        ax.hist(x, bins=n_bins, color=PASTEL[i // 4], edgecolor=EDGE, linewidth=.35)
        unidad = diccionario.set_index("nombre").loc[v, "unidad"]
        ax.set_title(v)
        ax.set_xlabel(unidad.replace(" (ventana", "\n(ventana"))
        ax.set_ylabel("Flujos")
        ax.xaxis.set_major_locator(MaxNLocator(nbins=4))
        ax.ticklabel_format(axis="x", style="sci", scilimits=(-3, 4), useMathText=True)
        ax.grid(axis="y", color="#E5E9EC", linewidth=.5)
    for ax in axes.flat[len(lista):]:
        ax.axis("off")
    fig.suptitle(titulo, fontsize=17)
    nota(fig, f"5.000 flujos · {escala} · rango completo, sin eliminar extremos · intervalos FD limitados a
    ↪ 10-80")
    guardar(fig, nombre)

histogramas(variables, "02_histogramas_log", "Distribuciones de X_num · escala log(x + 1)", "12 variables
    ↪ seleccionadas")
histogramas(crudas, "03_histogramas_crudos", "Distribuciones originales · comparación con la transformación
    ↪ ", "13 mediciones originales, incluido dpkts")

# Tamaño compacto: la figura va a media columna en el informe, así que la letra
# se dimensiona para que los valores se lean impresos (título y nota van en el epígrafe).
fig, ax = plt.subplots(figsize=(4.6, 4.0), layout="constrained")
cmap = LinearSegmentedColormap.from_list("pastel_divergente", ["#9CBAD4", "#FAFAF7", "#DFB0BC"])
im = ax.imshow(correlaciones.to_numpy(), vmin=-1, vmax=1, cmap=cmap)
labels = ["dur", "spkts", "sbytes", "dbytes", "sinpkt", "sjit", "synack",
```

```
          "srv_src", "srv_dst", "src_ltm", "src_dport", "dst_sport"]
labels = [dict(zip(["log_" + l for l in ["dur", "spkts", "sbytes", "dbytes", "sinpkt", "sjit", "synack",
          "ct_srv_src", "ct_srv_dst", "ct_src_ltm", "ct_src_dport_ltm", "ct_dst_sport_ltm"]], labels)).get(
    ↪ v, v)
          for v in variables]
ax.set_xticks(range(12), labels, rotation=55, ha="right", fontsize=8.5)
ax.set_yticks(range(12), labels, fontsize=8.5)
ax.spines[:].set_visible(False)
for i in range(12):
    for j in range(12):
        ax.text(j, i, f"{correlaciones.iloc[i, j]:.1f}".replace(".", ","), ha="center", va="center",
                fontsize=7.5, color="#283C49")
barra = fig.colorbar(im, ax=ax, shrink=.8)
barra.ax.tick_params(labelsize=8)
barra.set_label("r de Pearson", fontsize=8.5)
coma_decimal(fig)
guardar(fig, "04_correlaciones")

fig, axes = plt.subplots(2, 4, figsize=(17, 9), layout="constrained")
for i, (ax, v) in enumerate(zip(axes.flat, frecuencias.variable.unique())):
    f = frecuencias[frecuencias.variable.eq(v)].sort_values("frecuencia")
    # Mantener el orden natural de las bandas, aunque sus frecuencias sean similares.
    ordenes = {"duracion_banda": ["larga", "media", "corta"],
               "bytes_origen_banda": ["altos", "medios", "bajos"]}
    if v in ordenes:
        f = f.set_index("categoria").loc[ordenes[v]].reset_index()
    ax.barh(f.categoria, f.porcentaje, color=PASTEL[i % len(PASTEL)], edgecolor=EDGE, linewidth=.5)
    ax.set_xlim(0, 100)
    ax.set_xlabel("Porcentaje de flujos")
    ax.set_title(v)
    ax.grid(axis="x", color="#E5E9EC", linewidth=.5)
    for j, row in enumerate(f.itertuples()):
        ax.text(row.porcentaje + 1, j, f"{row.porcentaje:.1f}%".replace(".", ","), va="center", fontsize
    ↪ =9)
axes.flat[-1].axis("off")
fig.suptitle("Frecuencias de X_cat · propuesta operativa para ACM", fontsize=17)
nota(fig, "5.000 flujos · 4 categóricas existentes + 3 numéricas discretizadas · ninguna modalidad por
    ↪ debajo de 5 %")
guardar(fig, "05_frecuencias_categoricas")

fig, ax = plt.subplots(figsize=(10, 7), layout="constrained")
f = msa.sort_values("msa")
ax.barh(f.variable, f.msa, color=PASTEL[0], edgecolor=EDGE, linewidth=.5)
ax.axvline(.5, color="#7C5363", linestyle="--", linewidth=1.2, label="Umbral MSA = 0,50")
ax.set_xlim(0, 1)
for i, row in enumerate(f.itertuples()):
    ax.text(row.msa + .01, i, f"{row.msa:.3f}".replace(".", ","), va="center")
ax.set_xlabel("MSA por variable")
ax.set_title(f"Adecuación por variable · KMO global = {piloto.kmo_global:.3f}".replace(".", ","), fontsize
    ↪ =15)
ax.legend(loc="lower right")
guardar(fig, "06_msa")


print("Se guardaron las seis figuras en resultados/figuras.")
```

`python/graficos_parte2.py` · Gráficos de las Etapas 1 a 4

```
"""Figuras de todas las técnicas. Ejecutar después de R/ejecutar_todo.R."""
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.ticker import MaxNLocator, FuncFormatter, ScalarFormatter
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
T = ROOT / "resultados/tablas"
F = ROOT / "resultados/figuras"
```

```
F.mkdir(exist_ok=True)
def leer(n): return pd.read_csv(T / f"{n}.csv")
base = pd.read_excel(ROOT / "muestra_estratificada-simulado-de-ataques.xlsx", sheet_name="muestra")
vars_num = leer("conjuntos_congelados").query('conjunto == "X_num"').variable.tolist()
ABR = ["dur", "spkts", "sbytes", "dbytes", "sinpkt", "sjit", "synack", "srv_src", "srv_dst", "src_ltm", "
    ↪ src_dport", "dst_sport"]
COL = ["#B9D2E5", "#D1C3E0", "#BCD8C5", "#E6C4A7", "#E5BCC8", "#C7D8D4", "#D8D2B1"]
TRAFICO = dict(zip(["Normal", "Exploits", "Fuzzers", "Reconnaissance", "Otros ataques"], COL))
DARK = "#405361"
CMAP = LinearSegmentedColormap.from_list("suave", ["#94B7D2", "#FCFCF9", "#D9A7B5"])
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 13,
    "axes.spines.top": False, "axes.spines.right": False, "axes.titlesize": 13,
    "text.color": DARK, "axes.labelcolor": DARK, "figure.facecolor": "white"})

def coma_decimal(fig):
    """Usar coma decimal en los ejes numéricos, como en el resto del informe."""
    for ax in fig.axes:
        for eje in (ax.xaxis, ax.yaxis):
            if isinstance(eje.get_major_formatter(), ScalarFormatter):
                eje.set_major_formatter(FuncFormatter(lambda v, _: f"{v:g}".replace(".", ",")))

def guardar(fig, nombre):
    coma_decimal(fig)
    fig.savefig(F / f"{nombre}.png", dpi=220, bbox_inches="tight", facecolor="white")
    plt.close(fig)

def fuente(base):
    """Tamaño de letra base (pt) de la figura: textos, ejes, ticks y leyendas."""
    plt.rcParams.update({"font.size": base, "axes.titlesize": base, "axes.labelsize": base,
        "xtick.labelsize": base, "ytick.labelsize": base, "legend.fontsize": base})

def oscurecer(c, f=.3):
    """Mezcla el color con el gris oscuro del repo para ganar contraste sin cambiar el tono."""
    from matplotlib.colors import to_rgb
    return tuple((1 - f) * np.array(to_rgb(c)) + f * np.array(to_rgb(DARK)))

def origen(ax):
    ax.axhline(0, color="#D0D6DB", lw=.7)
    ax.axvline(0, color="#D0D6DB", lw=.7)

def adjust_text(texts, ax, expand=None, arrowprops=None, puntos=None, pad=7, peso=3):
    """Colocar etiquetas sin solaparlas (solo Matplotlib).

    Búsqueda por descenso de coordenadas: cada etiqueta prueba posiciones en anillos
    alrededor de su punto y se queda con la de menor costo (solapes con otras etiquetas,
    con marcadores y con líneas guía ajenas, más la distancia al punto).
    `puntos`: coordenadas de datos de obstáculos adicionales (cajas de lado 2*pad px).
    """
    ax.set_autoscale_on(False)
    ax.figure.canvas.draw()
    rend = ax.figure.canvas.get_renderer()
    ab = ax.bbox
    anclas = [np.array(ax.transData.transform(t.get_position())) for t in texts]
    obst = np.array(ax.transData.transform(np.asarray(puntos))) if puntos is not None else np.empty((0, 2))
    ang = np.deg2rad(np.arange(0, 360, 15))
    cands = [(0, 0)] + [(r * np.cos(a), r * np.sin(a)) for r in (12, 20, 30, 42, 56, 72, 92, 115, 140) for
    ↪ a in ang]
    def caja(t, i, dx, dy):
        h = "left" if dx > 3 else "right" if dx < -3 else "center"
        v = "bottom" if dy > 3 else "top" if dy < -3 else "center"
        t.set_ha(h); t.set_va(v)
        t.set_position(ax.transData.inverted().transform(anclas[i] + [dx, dy]))
        return t.get_window_extent(rend).expanded(1.08, 1.2)
    def guia(bb, p):
        q = np.array([min(max(p[0], bb.x0), bb.x1), min(max(p[1], bb.y0), bb.y1)])
        return (p, q) if np.hypot(*(q - p)) > 9 else None
    def cruza(a, b, c, d):
        o = lambda p, q, r: (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
        return o(a, b, c) * o(a, b, d) < 0 and o(c, d, a) * o(c, d, b) < 0
    def seg_en_caja(g, bb):
        return any(bb.x0 < x < bb.x1 and bb.y0 < y < bb.y1 for x, y in (g[0] + (g[1] - g[0]) * u for u in
    ↪ np.linspace(0, 1, 14)))
    def solape(a, b):
```

```
        w = min(a.x1, b.x1) - max(a.x0, b.x0); h = min(a.y1, b.y1) - max(a.y0, b.y0)
        return w * h if w > 0 and h > 0 else 0
    todos = np.vstack([obst, np.array(anclas)]) if len(obst) else np.array(anclas)
    OB = np.column_stack([todos[:, 0] - pad, todos[:, 1] - pad, todos[:, 0] + pad, todos[:, 1] + pad])
    n = len(texts)
    elec = [(0, 0)] * n
    cajas = [caja(t, i, 0, 0) for i, t in enumerate(texts)]
    def costo(i, c):
        bb = caja(texts[i], i, *c)
        if bb.x0 < ab.x0 or bb.x1 > ab.x1 or bb.y0 < ab.y0 or bb.y1 > ab.y1: return 1e9
        k = peso * np.hypot(*c)
        for j in range(n):
            if j != i: k += 120 * solape(bb, cajas[j])
        W = np.clip(np.minimum(bb.x1, OB[:, 2]) - np.maximum(bb.x0, OB[:, 0]), 0, None)
        H = np.clip(np.minimum(bb.y1, OB[:, 3]) - np.maximum(bb.y0, OB[:, 1]), 0, None)
        k += 80 * float(np.sum(W * H))
        g = guia(bb, anclas[i])
        if g is not None:
            for j in range(n):
                if j == i: continue
                if seg_en_caja(g, cajas[j]): k += 1500
                gj = guia(cajas[j], anclas[j])
                if gj is not None and cruza(*g, *gj): k += 1500
            if len(obst):
                P = g[0] + (g[1] - g[0]) * np.linspace(0, 1, 14)[:, None]
                d = np.min(np.hypot(P[:, None, 0] - obst[None, :, 0], P[:, None, 1] - obst[None, :, 1]),
    ↪ axis=1)
                k += 150 * float(np.sum(d < 4))
        return k
    orden = sorted(range(n), key=lambda i: np.sum(np.hypot(*(np.array(anclas) - anclas[i]).T) < 60),
    ↪ reverse=True)
    for _ in range(8):
        for i in orden:
            mejor = min(cands, key=lambda c: costo(i, c))
            elec[i] = mejor; cajas[i] = caja(texts[i], i, *mejor)
    inv = ax.transData.inverted()
    for i, t in enumerate(texts):
        cajas[i] = caja(t, i, *elec[i])
        g = guia(cajas[i], anclas[i])
        if g is not None:
            (x0, y0), (x1, y1) = inv.transform(g[0]), inv.transform(g[1])
            ax.plot([x0, x1], [y0, y1], color="#7F8A92", linewidth=.5, zorder=1)

def dendrogram(link, ax, labels=None, truncate_mode=None, p=25, orientation="top", **kwargs):
    """Dibujar con Matplotlib el árbol Ward ya calculado en R, sin recalcularlo."""
    n = len(link) + 1
    limite = 2 * n - p if truncate_mode else n
    hojas = []
    def ordenar(node):
        if node < n or node < limite:
            hojas.append(node)
        else:
            ordenar(int(link[node - n, 0])); ordenar(int(link[node - n, 1]))
    ordenar(2 * n - 2)
    posiciones = {node: i for i, node in enumerate(hojas)}
    def trazar(node):
        if node in posiciones: return posiciones[node], 0
        row = link[node - n]
        a, ha = trazar(int(row[0])); b, hb = trazar(int(row[1]))
        xx, yy = [a, a, b, b], [ha, row[2], row[2], hb]
        if orientation == "right": ax.plot(yy, xx, color="#8EA9BE", lw=1)
        else: ax.plot(xx, yy, color="#8EA9BE", lw=1)
        return (a + b) / 2, row[2]
    trazar(2 * n - 2)
    nombres = [(labels[node] if labels else str(node + 1)) if node < n else f"({int(link[node - n, 3])})"
    ↪ for node in hojas]
    if orientation == "right": ax.set_yticks(range(len(hojas)), nombres)
    else: ax.set_xticks(range(len(hojas)), nombres)

# Resumen gráfico de datos para el informe (Etapa 0): mapa de calor de correlaciones con
# valores, diagramas de caja de las 12 variables con los límites de Tukey y MSA por variable,
# alineados fila a fila. Ancho 11,5 pulgadas -> impreso a 16 cm (factor 0,55): letra de 14 pt
# = 7,7 pt impresos. Todo se lee de resultados/tablas; las figuras completas se conservan.
```

```
fuente(14)
lim = leer("atipicos_limites").set_index("variable")
corr = leer("correlaciones").set_index("variable").loc[vars_num, vars_num]
msa = leer("msa_por_variable").set_index("variable").loc[vars_num, "msa"]
piloto = leer("piloto_supuestos").iloc[0]
fig = plt.figure(figsize=(11.5, 5.0), layout="constrained")
rej = fig.add_gridspec(1, 3, width_ratios=[6.1, 2.6, 1.9])
ax1, ax2, ax3 = (fig.add_subplot(rej[0, i]) for i in range(3))
im = ax1.imshow(corr.to_numpy(), cmap=CMAP, vmin=-1, vmax=1, aspect="auto")
ax1.set_xticks(range(12), ABR, rotation=60, ha="right")
ax1.set_yticks(range(12), ABR)
ax1.spines[:].set_visible(False)
for i in range(12):
    for j in range(12):
        ax1.text(j, i, f"{corr.iloc[i, j]:.1f}".replace(".", ","), ha="center", va="center", fontsize=14)
ax1.set_title("Correlaciones de Pearson")
stats = []
for v in vars_num:
    r = lim.loc[v]; x = base[v]
    stats.append(dict(med=r.mediana, q1=r.q1_tukey, q3=r.q3_tukey, whislo=r.bigote_inferior,
        whishi=r.bigote_superior, fliers=x[(x < r.limite_inferior) | (x > r.limite_superior)].to_numpy()))
ax2.bxp(stats, positions=range(12), vert=False, widths=.6, patch_artist=True, manage_ticks=False,
    boxprops=dict(facecolor=COL[1], edgecolor=DARK), medianprops=dict(color=DARK, lw=1.5),
    whiskerprops=dict(color=DARK), capprops=dict(color=DARK),
    flierprops=dict(marker=".", markersize=2.5, alpha=.25, markeredgecolor=DARK, markerfacecolor=DARK))
ax2.set_ylim(11.5, -.5); ax2.set_yticks([]); ax2.set_xlim(-.5, 17)
ax2.set_xticks([0, 5, 10, 15]); ax2.set_xlabel("log(x + 1)")
ax2.set_title("Cajas (Tukey)")
for i, v in enumerate(vars_num):
    ax2.text(1.03, i, f"{int(lim.loc[v, 'n_atipicos']):,}".replace(",", "."), transform=ax2.
    ↪ get_yaxis_transform(),
        va="center", ha="left", fontsize=13)
ax3.barh(range(12), msa.to_numpy(), color=COL[0], edgecolor=DARK, lw=.5)
ax3.axvline(.5, color="#7C5363", ls="--", lw=1.2)
ax3.set_ylim(11.5, -.5); ax3.set_yticks([]); ax3.set_xlim(0, 1.3); ax3.set_xticks([0, .5, 1])
for i, m in enumerate(msa):
    ax3.text(m + .03, i, f"{m:.2f}".replace(".", ","), va="center", fontsize=13)
ax3.set_xlabel("MSA")
ax3.set_title(f"MSA (KMO {piloto.kmo_global:.3f})".replace(".", ","))
guardar(fig, "07_datos_resumen")

# Histogramas de seis variables de X_num (dos por bloque, escala log), compactos para el cuerpo del informe;
# los de las 12 variables están en 02_histogramas_log.png.
sel = ["log_spkts", "log_dbytes", "log_dur", "log_sjit", "log_ct_srv_src", "log_ct_dst_sport_ltm"]
fig, axes = plt.subplots(2, 3, figsize=(11.5, 3.9), layout="constrained")
for ax, v in zip(axes.flat, sel):
    ax.hist(base[v], bins=30, color=COL[1], edgecolor=DARK, lw=.3)
    ax.set_title(ABR[vars_num.index(v)])
    ax.xaxis.set_major_locator(MaxNLocator(nbins=4))
    ax.yaxis.set_major_locator(MaxNLocator(nbins=3))
for ax in axes[:, 0]: ax.set_ylabel("Flujos")
guardar(fig, "07b_histogramas_log")
fuente(13)

# ACP: el biplot usa escalas recíprocas, conservando el producto fila-columna.
eig = leer("acp_autovalores")
scores = leer("acp_puntajes")
r = leer("acp_correlaciones").set_index("variable")
fuente(10.5)  # figura de 7,0 in -> impresa a 13,6 cm (x0,76): letra impresa ~8 pt
fig, axs = plt.subplots(1, 2, figsize=(7.0, 3.1), layout="constrained", gridspec_kw={"width_ratios": [1,
    ↪ 1.7]})
axs[0].plot(eig.componente, eig.autovalor, "o-", color="#8AABC5", markeredgecolor=DARK)
axs[0].axhline(1, color=DARK, ls="--", lw=.8)
axs[0].set(title="Codo del ACP", xlabel="Componente", ylabel="Autovalor", xticks=[1, 2, 3, 4, 6, 8, 10,
    ↪ 12])
ax = axs[1]
sx = np.sqrt(eig.autovalor.iloc[:2].to_numpy())
xy = .5 * scores[["PC1", "PC2"]].to_numpy() / sx
for cat, color in TRAFICO.items():
    sel = scores.tipo_trafico.eq(cat)
    ax.scatter(xy[sel, 0], xy[sel, 1], s=5, color=oscurecer(color, .3), alpha=.6, linewidths=0, label=cat,
    ↪ rasterized=True)
texts = []
```

```
ax.set_xlim(-2.35, 2.35)
ax.set_ylim(-1.5, 2.1)
for i, v in enumerate(vars_num):
    arrow = 2 * r.loc[v, ["PC1", "PC2"]].to_numpy()
    ax.annotate("", xy=arrow, xytext=(0, 0), arrowprops=dict(arrowstyle="->", color=DARK, lw=.8))
    texts.append(ax.text(arrow[0], arrow[1], ABR[i], bbox=dict(boxstyle="round,pad=.1", fc="white", ec="
    ↪ none", alpha=.7), zorder=5))
ext = leer("acp_extremos")
for row in ext.itertuples():
    j = scores.index[scores.id.eq(row.id)][0]
    ax.scatter(*xy[j], s=26, facecolor="none", edgecolor=DARK, linewidth=.9, zorder=4)
    texts.append(ax.text(*xy[j], str(row.id), fontweight="bold", bbox=dict(boxstyle="round,pad=.12", fc="
    ↪ white", ec="none", alpha=.8)))
origen(ax)
pct = eig.porcentaje.iloc[:2].round(2).map(lambda v: f"{v:.2f}".replace(".", ","))
ax.set(title="Biplot de individuos y variables", xlabel=f"CP1 ({pct.iloc[0]}%)", ylabel=f"CP2 ({pct.iloc
    ↪ [1]}%)")
from matplotlib.lines import Line2D
handles = [Line2D([], [], marker="o", ls="", markersize=7, markerfacecolor=oscurecer(c, .3),
    ↪ markeredgecolor=DARK, markeredgewidth=.5) for c in TRAFICO.values()]
fig.legend(handles, list(TRAFICO), loc="outside lower center", ncol=5, columnspacing=1.0, handletextpad=.2)
adjust_text(texts, ax=ax, arrowprops=dict(arrowstyle="-", color="#929BA2", lw=.4),
    puntos=np.vstack([u * 2 * r.loc[v, ["PC1", "PC2"]].to_numpy() for v in vars_num for u in np.linspace
    ↪ (.35, 1, 24)]
        + [xy[scores.index[scores.id.eq(i)][0]][None] for i in ext.id]), pad=2, peso=25)
guardar(fig, "08_acp_codo_biplot")

par = leer("af_paralelo")
fuente(10)  # 6,5 in -> 13,6 cm (x0,82): ~8,2 pt
fig, ax = plt.subplots(figsize=(6.5, 2.3), layout="constrained")
ax.plot(par.dimension, par.factor_observado, "o-", color="#8AABC5", label="Factores observados")
ax.plot(par.dimension, par.factor_simulado, "s--", color="#B7A4C9", label="Referencia aleatoria")
ax.axvline(3.5, color=DARK, ls=":", lw=1, label="Factores retenidos (3, Kaiser reducido)")
ax.axvline(4.5, color="#929BA2", ls="--", lw=.8, label="Corte del análisis paralelo (4)")
ax.set(xlabel="Número de factor", ylabel="Autovalor reducido", title="Análisis paralelo · 100 repeticiones",
    ↪  xticks=range(1, 13))
ax.set_ylim(-0.8, 7.6)
ax.legend(ncol=2, loc="upper right", columnspacing=1.0, handlelength=1.6)
guardar(fig, "09_af_paralelo")

def linkage_r(nombre, n):
    d = leer(nombre); counts = np.zeros(n - 1); rows = []
    for i, row in enumerate(d.itertuples()):
        a = -int(row.izquierda) - 1 if row.izquierda < 0 else n + int(row.izquierda) - 1
        b = -int(row.derecha) - 1 if row.derecha < 0 else n + int(row.derecha) - 1
        count = (1 if a < n else counts[a - n]) + (1 if b < n else counts[b - n])
        counts[i] = count
        rows.append([a, b, row.altura, count])
    return np.array(rows)

metricas = leer("cluster_metricas")
k = int(leer("cluster_comparaciones").iloc[0].k)
fuente(10.5)  # 6,5 in -> 12,8 cm (x0,775): ~8,1 pt
fig, axs = plt.subplots(1, 2, figsize=(6.5, 2.0), layout="constrained")
for ax, pref, titulo, ylabel in [(axs[0], "wss", "Codo", "Suma de cuadrados interna"),
    (axs[1], "silhouette", "Silhouette promedio", "Silhouette")]:
    ax.plot(metricas.k, metricas[f"{pref}_ward"], "s--", color="#B7A4C9", label="Ward")
    ax.plot(metricas.k, metricas[f"{pref}_kmeans"], "o-", color="#8AABC5", label="K-means")
    ax.axvline(k, color=DARK, lw=.7, ls=":")
    ax.set(title=titulo, xlabel="k", ylabel=ylabel, xticks=range(2, 9))
axs[0].legend(loc="upper right")
guardar(fig, "10_cluster_seleccion")

fuente(13)
fig, ax = plt.subplots(figsize=(10, 2.6), layout="constrained")
link = linkage_r("cluster_dendrograma", len(base))
dendrogram(link, truncate_mode="lastp", p=15, ax=ax, link_color_func=lambda _: "#8EA9BE", leaf_font_size=8)
ax.axhline((link[-k, 2] + link[-k + 1, 2]) / 2, color=DARK, ls="--", lw=.8)
ax.set(title="Ward.D2 · últimos 15 nodos del árbol de 5.000 flujos", xlabel="Nodos agregados (tamaño entre
    ↪ paréntesis)", ylabel="Altura")
guardar(fig, "11_cluster_dendrograma")

fuente(11)  # 8,0 in -> 14,1 cm (x0,69): ~7,6 pt
```

```
fig, axs = plt.subplots(1, 2, figsize=(8.0, 2.9), layout="constrained", gridspec_kw={"width_ratios": [2.0, 1]})
perfil = leer("cluster_medias_estandarizadas").set_index("cluster")
im = axs[0].imshow(perfil, cmap=CMAP, vmin=-3, vmax=3, aspect="auto")
axs[0].set_xticks(range(12), ABR, rotation=55, ha="right")
axs[0].set_yticks(range(k), [f"C{i}" for i in range(1, k + 1)])
axs[0].set_title("Medias z por cluster")
for i in range(k):
    for j in range(12): axs[0].text(j, i, f"{perfil.iloc[i,j]:.1f}".replace(".", ","), ha="center", va="center", fontsize=11)
fig.colorbar(im, ax=axs[0], shrink=.65, label="z (satura en ±3)")
dendrogram(linkage_r("cluster_dendrograma_variables", 12), labels=ABR, orientation="right",
    ax=axs[1], link_color_func=lambda _: "#8EA9BE", leaf_font_size=10)
axs[1].set(title="Ward de variables", xlabel="Distancia")
guardar(fig, "12_cluster_perfiles_variables")

fuente(11.5)  # 5,2 in -> 8,8 cm (x0,67): ~7,7 pt
fig, ax = plt.subplots(figsize=(5.2, 2.15), layout="constrained")  # compacta: va a media página
texts = []
coords_acs = []
for name, color, marker in [("acs_filas", COL[0], "o"), ("acs_columnas", COL[4], "s")]:
    d = leer(name)
    coords_acs.append(d.iloc[:, 1:3].to_numpy())
    ax.scatter(d.iloc[:,1], d.iloc[:,2], color=color, edgecolor=DARK, s=50, marker=marker,
        label="Tipo de tráfico" if name.endswith("filas") else "Estado")
    # Con solo ocho puntos, cada etiqueta se ubica con un desplazamiento fijo
    # (en puntos) para que no tape su marcador ni a sus vecinas.
    lugar = {"Otros ataques": (0, 7, "center"), "INT": (0, -13, "center"),
             "Fuzzers": (0, 7, "center"), "Reconnaissance": (0, -13, "center"),
             "FIN": (8, 0, "left"), "Exploits": (8, 0, "left"),
             "Normal": (8, 0, "left"), "CON_otros": (0, -13, "center")}
    for _, row in d.iterrows():
        dx, dy, ha = lugar.get(row.iloc[0], (6, 6, "left"))
        ax.annotate(row.iloc[0], (row.iloc[1], row.iloc[2]), xytext=(dx, dy),
                    textcoords="offset points", ha=ha, va="center")
coords_acs = np.vstack(coords_acs)
ax.set_xlim(coords_acs[:, 0].min() - .4, coords_acs[:, 0].max() + .25)
ax.set_ylim(coords_acs[:, 1].min() - .2, coords_acs[:, 1].max() + .45)
origen(ax)
ax.set(xlabel="Dimensión 1", ylabel="Dimensión 2", title="ACS · tipo de tráfico y estado")
ax.legend(loc="upper center", bbox_to_anchor=(0.4, 1), ncol=2, columnspacing=1.0, handletextpad=.2)
guardar(fig, "13_acs_mapa")

e = leer("acm_autovalores")
fuente(10)  # 6,5 in -> ~13,6 cm (x0,82): ~8,2 pt
fig, axs = plt.subplots(1, 2, figsize=(6.5, 1.9), layout="constrained")
axs[0].plot(e.dimension, e.autovalor, "o-", color="#8AABC5")
axs[0].axhline(1/leer("acm_resumen").Q.iloc[0], color=DARK, ls="--", lw=.8)
axs[0].set(title="Autovalores crudos", xlabel="Dimensión", ylabel="Autovalor")
axs[0].xaxis.set_major_locator(MaxNLocator(integer=True))
axs[1].bar(e.dimension.iloc[:6], e.porcentaje_ajustado.iloc[:6], color=COL[1], edgecolor=DARK, linewidth=.5)
axs[1].set(title="Inercia ajustada", xlabel="Dimensión", ylabel="Porcentaje")
guardar(fig, "14_acm_inercia")

co = leer("acm_categorias_coordenadas")
variables_cat = co.categoria.str.split(":").str[0].unique().tolist()
prefijos = dict(zip(variables_cat, ["Tr", "Pr", "Sv", "Es", "Rp", "Du", "By"]))
marcas = ["o", "s", "^", "D", "v", "P", "X"]
fuente(11.5)  # 6,5 in -> 10,9 cm (x0,66): ~7,6 pt
fig, ax = plt.subplots(figsize=(6.5, 3.9), layout="constrained")
texts = []
for i, var in enumerate(variables_cat):
    d = co[co.categoria.str.startswith(var + ":")]
    ax.scatter(d["Dim.1"], d["Dim.2"], color=COL[i], edgecolor=DARK, linewidth=.6,
        marker=marcas[i], s=45, label=f"{prefijos[var]}: {var}")
    for row in d.itertuples(index=False, name=None):
        lab = row[0].split(":", 1)[1].replace("Otros ataques", "Otros at.").replace("Reconnaissance", "Recon.")
        texts.append(ax.text(row[1], row[2], prefijos[var] + ":" + lab))
ax.margins(.22)
origen(ax)
```

```
ax.set(title="ACM · categorías activas", xlabel="Dimensión 1 · 27,42% cruda", ylabel="Dimensión 2 · 14,03% cruda")
fig.supxlabel("Tr tráfico · Pr protocolo · Sv servicio · Es estado\nRp respuesta · Du duración · By bytes", fontsize=11.5)
adjust_text(texts, ax=ax, expand=(1.1, 1.2), arrowprops=dict(arrowstyle="-", color="#AAB2B9", lw=.5),
    puntos=co[["Dim.1", "Dim.2"]].to_numpy(), pad=6, peso=8)
guardar(fig, "15_acm_categorias")

fuente(10)  # 6,5 in -> ~13,6 cm (x0,82): ~8,2 pt
fig, axs = plt.subplots(1, 2, figsize=(6.5, 2.6), layout="constrained")
ind = leer("acm_individuos")
su = leer("acm_cluster_suplementario")
for i in range(1, k + 1):
    d = ind[ind.cluster.eq(i)]
    axs[0].scatter(d["Dim.1"], d["Dim.2"], s=5, alpha=.15, color=COL[i - 1], rasterized=True)
    r = su[su.cluster.eq(i)].iloc[0]
    axs[0].scatter(r["Dim.1"], r["Dim.2"], color=COL[i - 1], edgecolor=DARK, marker="D", s=65)
    axs[0].annotate(f"C{i}", (r["Dim.1"], r["Dim.2"]), xytext={2: (0, 11), 4: (10, -3)}.get(i, (6, 4)), ha="center" if i == 2 else "left", va="center" if i == 4 else "baseline", textcoords="offset points")
origen(axs[0])
axs[0].set(title="Clusters suplementarios", xlabel="Dimensión 1", ylabel="Dimensión 2")
eta = leer("acm_eta2")
y = np.arange(len(eta))
axs[1].barh(y - .18, eta["Dim.1"], height=.35, color=COL[0], edgecolor=DARK, lw=.4, label="Dim 1")
axs[1].barh(y + .18, eta["Dim.2"], height=.35, color=COL[1], edgecolor=DARK, lw=.4, label="Dim 2")
axs[1].set_yticks(y, eta.variable)
axs[1].set(xlim=(0, 1), xlabel="eta²", title="Discriminación por variable")
axs[1].set_ylim(-.6, len(eta) + .45)
axs[1].legend(loc="upper right", ncol=2, columnspacing=1.0, handlelength=1.2)
guardar(fig, "16_acm_clusters_eta")
print("Figuras 07 a 16 generadas con Matplotlib.")
```

`python/graficos_logistica.py` · Gráficos de la regresión logística

```
"""Figuras 17 y 18 de la regresión logística. Ejecutar después de R/06_logistica.R."""
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.ticker import FuncFormatter, ScalarFormatter
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
T = ROOT / "resultados/tablas"
F = ROOT / "resultados/figuras"
F.mkdir(exist_ok=True)
def leer(n): return pd.read_csv(T / f"log_reg_{n}.csv")

# Mismo estilo que graficos_parte2.py
COL = ["#B9D2E5", "#D1C3E0", "#BCD8C5", "#E6C4A7", "#E5BCC8", "#C7D8D4", "#D8D2B1"]
DARK = "#405361"
CMAP = LinearSegmentedColormap.from_list("suave", ["#FCFCF9", "#94B7D2"])
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 13,
    "axes.spines.top": False, "axes.spines.right": False, "axes.titlesize": 13,
    "text.color": DARK, "axes.labelcolor": DARK, "figure.facecolor": "white"})

def coma_decimal(fig):
    """Usar coma decimal en los ejes numéricos, como en el resto del informe."""
    for ax in fig.axes:
        for eje in (ax.xaxis, ax.yaxis):
            if isinstance(eje.get_major_formatter(), ScalarFormatter):
                eje.set_major_formatter(FuncFormatter(lambda v, _: f"{v:g}".replace(".", ",")))

def guardar(fig, nombre):
    coma_decimal(fig)
    fig.savefig(F / f"{nombre}.png", dpi=220, bbox_inches="tight", facecolor="white")
    plt.close(fig)

roc = leer("roc_curva")
```

```
auc = leer("auc").iloc[0]
conf = leer("confusion")
met = leer("metricas").iloc[0]

# Figura 17: ROC en prueba + matriz de confusión (umbral 0,5)
# Tamaño compacto: en el informe va a 0,6 del ancho de texto (9,6 cm); con esta
# figura de 13,2 cm la letra impresa queda en unos 7,5 pt.
plt.rcParams.update({"font.size": 10.5})
fig, axs = plt.subplots(1, 2, figsize=(5.2, 2.3), layout="constrained")
ax = axs[0]
ax.plot([0, 1], [0, 1], ls="--", color="#AAB2B9", lw=1, label="Azar (AUC = 0,500)")
ax.plot(roc.fpr, roc.tpr, color="#6F9BBC", lw=2.2,
        label=f"Modelo (AUC = {auc.AUC:.3f})".replace(".", ","))
ax.fill_between(roc.fpr, roc.tpr, alpha=.15, color=COL[0])
punto = (1 - met.especificidad, met.sensibilidad)
ax.scatter(*punto, s=30, color="#D9A7B5", edgecolor=DARK, zorder=3, label="Umbral 0,5")
ax.set(xlabel="1 - especificidad", ylabel="Sensibilidad",
       xlim=(0, 1), ylim=(0, 1.01), aspect="equal")
# Sin leyenda (no entra a este tamaño): el AUC va rotulado y el epígrafe del
# informe explica la diagonal de azar y el punto del umbral 0,5.
ax.text(.97, .05, f"AUC = {auc.AUC:.3f}".replace(".", ","), transform=ax.transAxes,
        ha="right", va="bottom", fontsize=10.5, color=DARK, fontweight="bold")
ax.annotate("umbral 0,5", punto, xytext=(8, -12), textcoords="offset points", fontsize=9.5, color=DARK)

ax = axs[1]
m = np.array([[conf.query('real=="normal" and predicho=="normal"').n.iloc[0],
               conf.query('real=="normal" and predicho=="ataque"').n.iloc[0]],
              [conf.query('real=="ataque" and predicho=="normal"').n.iloc[0],
               conf.query('real=="ataque" and predicho=="ataque"').n.iloc[0]]])
pct = m / m.sum(axis=1, keepdims=True)
ax.imshow(pct, cmap=CMAP, vmin=0, vmax=1)
for i in range(2):
    for j in range(2):
        ax.text(j, i, f"{m[i, j]}\n({100 * pct[i, j]:.1f}%)".replace(".", ","), ha="center", va="center",
                fontsize=11, color=DARK, fontweight="bold" if i == j else "normal")
ax.set_xticks([0, 1], ["Normal", "Ataque"])
ax.set_yticks([0, 1], ["Normal", "Ataque"])
ax.set(xlabel="Clase predicha", ylabel="Clase real")
for s in ax.spines.values():
    s.set_visible(False)
ax.tick_params(length=0)
guardar(fig, "17_logistica_roc_confusion")
plt.rcParams.update({"font.size": 13})

# Figura 18: forest plot de OR (escala log) + calibración por deciles
co = leer("coeficientes").set_index("termino")
orx = leer("or_por_desvio").set_index("termino")
nombres = {"PC1": "CP1 (por desvío)", "PC2": "CP2 (por desvío)", "PC3": "CP3 (por desvío)",
           "proto_gudp": "Protocolo: udp (vs tcp)", "proto_gotros": "Protocolo: otros (vs tcp)",
           "service_gdns": "Servicio: dns (vs ninguno)", "service_ghttp": "Servicio: http (vs ninguno)",
           "service_gotros": "Servicio: otros (vs ninguno)",
           "state_gCON_otros": "Estado: CON_otros (vs FIN)", "state_gINT": "Estado: INT (vs FIN)"}
filas = []
for t in nombres:
    if t not in co.index: continue
    if t in orx.index:
        r = orx.loc[t]; filas.append((nombres[t], r.OR_por_desvio, r.OR_desvio_IC95_inf, r.
    ↪ OR_desvio_IC95_sup, t))
    else:
        r = co.loc[t]; filas.append((nombres[t], r.OR, r.OR_IC95_inf, r.OR_IC95_sup, t))
filas = filas[::-1]
fig, axs = plt.subplots(1, 2, figsize=(10.5, 4.6), layout="constrained", gridspec_kw={"width_ratios":
    ↪ [1.25, 1]})
ax = axs[0]
y = np.arange(len(filas))
for i, (n, o, lo, hi, t) in enumerate(filas):
    color = "#6F9BBC" if t.startswith("PC") else "#C9899B"
    ax.plot([lo, hi], [i, i], color=color, lw=2)
    ax.scatter(o, i, color=color, edgecolor=DARK, s=40, zorder=3)
ax.axvline(1, color=DARK, lw=.9, ls="--")
ax.set_xscale("log")
ax.set_xticks([.05, .2, 1, 5, 20, 100, 400], ["0,05", "0,2", "1", "5", "20", "100", "400"])
ax.minorticks_off()
```

```
ax.set_yticks(y, [f[0] for f in filas], fontsize=11)
ax.set(xlabel="Odds ratio e IC 95 % (escala log)")

ax = axs[1]
cal = leer("calibracion")
ax.plot([0, 1], [0, 1], ls="--", color="#AAB2B9", lw=1, label="Calibración perfecta")
ax.plot(cal.p_media, cal.prop_observada, "o-", color="#6F9BBC", markeredgecolor=DARK, label="Deciles en prueba")
ax.set(xlabel="Probabilidad media predicha", ylabel="Proporción observada de ataques",
       xlim=(0, 1), ylim=(0, 1.02), aspect="equal")
ax.legend(loc="upper left", fontsize=11, frameon=False)
guardar(fig, "18_logistica_or_calibracion")
print("Figuras 17 y 18 generadas con Matplotlib.")
```

`python/verificar_resultados.py` · Verificación independiente de resultados

```
"""Comprobación independiente de los resultados de R con NumPy y pandas."""
from pathlib import Path
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
T = ROOT / "resultados/tablas"
def leer(n): return pd.read_csv(T / f"{n}.csv")
base = pd.read_excel(ROOT / "muestra_estratificada-simulado-de-ataques.xlsx", sheet_name="muestra")
assert base.shape == (5000, 68)
variables = leer("conjuntos_congelados").query('conjunto == "X_num"').variable.tolist()
x = base[variables].to_numpy()
z = (x - x.mean(axis=0)) / x.std(axis=0, ddof=1)
R = np.corrcoef(x, rowvar=False)
eig = np.linalg.eigvalsh(R)[::-1]
np.testing.assert_allclose(eig, leer("acp_autovalores").autovalor, atol=1e-12)
np.testing.assert_allclose(R, leer("correlaciones").iloc[:, 1:], atol=1e-12)
inversa = np.linalg.inv(R)
anti = inversa / np.sqrt(np.outer(inversa.diagonal(), inversa.diagonal()))
r2 = R**2; q2 = anti**2
np.fill_diagonal(r2, 0); np.fill_diagonal(q2, 0)
np.testing.assert_allclose(r2.sum() / (r2.sum() + q2.sum()), leer("piloto_supuestos").kmo_global.iloc[0])

asignaciones = leer("cluster_asignaciones")
np.testing.assert_array_equal(asignaciones.id, base.id)
wss = 0
for g in sorted(asignaciones.cluster.unique()):
    grupo = z[asignaciones.cluster.eq(g)]
    wss += ((grupo - grupo.mean(axis=0))**2).sum()
np.testing.assert_allclose(wss, leer("cluster_metricas").query('k == 6').wss_kmeans.iloc[0])
assert leer("cluster_tamanos").n.sum() == 5000
af = leer("af_cargas_completas")
# El AF final es GLS con 3 factores y se calcula por optimización: se admite su tolerancia numérica.
np.testing.assert_allclose((af[["F1", "F2", "F3"]]**2).sum(axis=1), af.comunalidad, atol=1e-4)
assert (af.unicidad > .005).all()  # la solución final no es un caso Heywood
assert (af[["F1", "F2", "F3"]].abs() >= .4).sum(axis=1).ge(1).all()
diag = leer("af_diagnostico_minres").set_index("metodo")
assert diag.loc["minres", "heywood"]  # el caso Heywood de minres se documenta, no se oculta
assert not diag.loc["gls", "heywood"]
# Las cargas deben reproducir la matriz de correlaciones con el RMSR informado.
L = af[["F1", "F2", "F3"]].to_numpy()
resid = R - L @ L.T
rmsr = np.sqrt(np.mean(resid[np.tril_indices(12, -1)]**2))
np.testing.assert_allclose(rmsr, leer("af_ajuste").rmsr.iloc[0], atol=1e-4)

# Rehacer el ACM indicador y el ajuste de Greenacre sin FactoMineR ni ca.
q = base[["tipo_trafico", "proto_g", "service_g", "state_g"]].copy()
q["respuesta_destino"] = np.where(base.dpkts.eq(0), "sin_respuesta", "con_respuesta")
c = leer("cortes_discretizacion").set_index("variable")
q["duracion_banda"] = pd.cut(base.dur, [-np.inf, c.loc["dur", "corte_1"], c.loc["dur", "corte_2"], np.inf],
     labels=["corta", "media", "larga"])
q["bytes_origen_banda"] = pd.cut(base.sbytes, [-np.inf, c.loc["sbytes", "corte_1"], c.loc["sbytes", "corte_2"], np.inf], labels=["bajos", "medios", "altos"])
A = pd.get_dummies(q, dtype=float).to_numpy()
n, Q, J = len(q), len(q.columns), A.shape[1]
```

```
P = A / (n * Q)
cm = P.sum(axis=0)
S = (P - cm / n) / np.sqrt(cm / n)
lamb = np.linalg.svd(S, compute_uv=False)[:J - Q]**2
acm = leer("acm_autovalores")
np.testing.assert_allclose(lamb, acm.autovalor, atol=1e-11)
ajustados = np.where(lamb > 1 / Q, (Q / (Q - 1) * (lamb - 1 / Q))**2, 0)
total = Q / (Q - 1) * (np.sum(lamb**2) - (J - Q) / Q**2)
np.testing.assert_allclose(ajustados, acm.autovalor_ajustado, atol=1e-11)
np.testing.assert_allclose(100 * ajustados / total, acm.porcentaje_ajustado, atol=1e-10)
t = leer("acs_contingencia").iloc[:, 1:].to_numpy()
esperados = np.outer(t.sum(axis=1), t.sum(axis=0)) / t.sum()
np.testing.assert_allclose(((t - esperados)**2 / esperados).sum(), leer("acs_asociacion").chi2.iloc[0])
assert (leer("frecuencias_X_cat").groupby("variable").frecuencia.sum() == 5000).all()
print("Verificación correcta: ACP, KMO, WSS, AF, ACS y ACM con ajuste de inercia.")
```